VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

TRIEU NGOC VAN ANH – 524H0005

DUONG TRAN QUOC KHANH – 523H0040

NGUYEN VO THANH TU – 523H0189

DESIGN, IMPLEMENT, AND AUDIT SECURE ACCESS CONTROL FOR A UNIVERSITY RECORDS SYSTEM

GROUP ASSIGNMENT REPORT

INTRODUCTION TO INFORMATION SECURITY

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY



TRIEU NGOC VAN ANH – 524H0005

DUONG TRAN QUOC KHANH – 523H0040

NGUYEN VO THANH TU – 523H0189

DESIGN, IMPLEMENT, AND AUDIT SECURE ACCESS CONTROL FOR A UNIVERSITY RECORDS SYSTEM

GROUP ASSIGNMENT REPORT

INTRODUCTION TO INFORMATION SECURITY

Advised by

Ph.D Huynh Ngoc Tu

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Ph.D Huynh Ngoc Tu, our instructor, for his valuable guidance throughout this group assignment on designing, implementing and auditing secure access control for the Introduction to Information Security course. His lectures on access control, software security and database security helped us turn security principles into concrete, testable controls.

We would also like to thank the Faculty of Information Technology, Ton Duc Thang University, for providing a supportive learning environment and equipping us with the foundational knowledge necessary to complete this assignment.

Due to limitations in time and knowledge, this report may inevitably contain certain shortcomings. We sincerely welcome any comments and suggestions from our instructor to further improve and refine our work.

Thank you sincerely.

Ho Chi Minh City, November 2026

Authors

(Signature and full name)

Trieu Ngoc Van Anh	Duong Tran Quoc Khanh	Nguyen Vo Thanh Tu

DECLARATION OF AUTHORSHIP

We hereby declare that this report and the accompanying source code are our own work, prepared under the guidance of Ph.D Huynh Ngoc Tu. The system design, implementation, test results and evidence presented in this report were produced by our group, and all test results were obtained by actually running the test suite against the implemented system. All external sources used are cited in the references.

In the event of any violation or misconduct, we accept full responsibility for the content of this report. Ton Duc Thang University shall not be held responsible for any intellectual property or copyright infringement committed by the team during the preparation of this report, if any.

Ho Chi Minh City, November 2026

Authors

(Signature and full name)

Trieu Ngoc Van Anh	Duong Tran Quoc Khanh	Nguyen Vo Thanh Tu

TABLE OF CONTENTS

LIST OF FIGURES vii
LIST OF TABLES x
LIST OF ABBREVIATIONS xi
ABSTRACT xii
CHAPTER 1. INTRODUCTION 1
1.1 Background and Objectives 1
1.2 Scope and Group Organization 1
CHAPTER 2. SYSTEM OVERVIEW 1
CHAPTER 3. THREAT MODEL AND SECURITY REQUIREMENTS 2
3.1 Assets, Users and Trust Boundaries 2
3.2 Threat-Model Table 3
CHAPTER 4. ACCESS-CONTROL POLICY 4
4.1 Access-Control Matrix 4
4.2 Policy Explanation 5
CHAPTER 5. SECURITY ARCHITECTURE AND IMPLEMENTATION 6
5.1 Authentication and Session Handling 6
5.2 Authorization 7
5.3 Grading and Class Management 7
5.4 Input Validation and Output Encoding 7
5.5 Security Audit Logging 8
5.6 Error Handling and Secrets 8
CHAPTER 6. DATABASE-SECURITY CONTROLS 8
CHAPTER 7. SECURITY TESTING 9
7.1 Test Plan 9
7.2 Results 9
CHAPTER 8. REMAINING RISKS AND IMPROVEMENTS 9
CHAPTER 9. CONCLUSION 10
INDIVIDUAL CONTRIBUTION SUMMARY 11
APPENDIX A. SYSTEM DEMONSTRATION 12
A.1 Interface Design 12
A.2 Authentication 12
A.3 Self-Registration and Class Catalog 14
A.4 Student Portal 17
A.5 Lecturer Workspace 20
A.6 Registrar Workspace 23

A.7 Administrator Console 27
A.8 Account Self-Service and Password Reset 29
A.9 Rejected Operations 33
A.10 Input Handling and Session Termination 36
A.11 Security Auditor Console 37
A.12 Database and Test Evidence 41
APPENDIX B. COMPLETE SECURITY TEST RESULTS **42**
APPENDIX C. REQUIREMENTS TRACEABILITY AND DEMO ACCOUNTS **48**
REFERENCES **50**

LIST OF FIGURES

Figure 3.1. System and data-flow diagram with trust boundaries TB1–TB3 and data stores D1–D4. 2

Figure A.1. Sign-in page with the product overview and the login form. 13

Figure A.2. Wrong password: one generic error message (HTTP 401) (T02). 13

Figure A.3. Disabled account with the correct password is rejected (T03). 14

Figure A.4. SQL-injection payload in the login form: generic failure, no database error (T18). 14

Figure A.5. Student self-registration form. 15

Figure A.6. Registration rejected by server-side validation (T46). 15

Figure A.7. Account created with the STUDENT role only; the user can sign in (T45). 16

Figure A.8. Dashboard of the newly registered student (new student code). 16

Figure A.9. Class catalog: the new student joins AI305 (seat counter updated) (T51). 17

Figure A.10. Student dashboard: study statistics and quick actions (T01). 18

Figure A.11. Student profile with the self-service contact form. 18

Figure A.12. Student's own enrollments. 19

Figure A.13. Own grades: four components, course totals and the credit-weighted overall average (T06). 19

Figure A.14. Class catalog seen by student01 (enrolled, open and full classes). 20

Figure A.15. Lecturer dashboard: assigned classes, students and incomplete totals. 21

Figure A.16. Lecturer sees only the classes assigned to them. 21

Figure A.17. Roster of IS101 with the four grade components and totals (T09). 22

Figure A.18. Components saved for S2026002; the database computes the total 8.00 (confirmation toast) (T10). 22

Figure A.19. Multi-role ta01 teaches SE202 but cannot grade their own enrollment (T27). 23

Figure A.20. Lecturer catalog: an unassigned class can be taken over (T52)....23

Figure A.21. Registrar dashboard: records, enrollments and classes....24

Figure A.22. Student search (the search text is a bound SQL parameter). 24

Figure A.23. Form for creating a student record....25

Figure A.24. All classes with lecturer and seat usage....25

Figure A.25. Creating a class: capacity at most 30, one optional lecturer....26

Figure A.26. Managing class SE310: lecturer assignment, adding students, seat usage (T48). 26

Figure A.27. Registrar enrolls S2026003 in IS101 (T12). 27

Figure A.28. Administrator dashboard: account statistics only, no academic data. 28

Figure A.29. User accounts, role chips, status and role-assignment controls (T16). 29

Figure A.30. My account (student): change name and password (T53). 30

Figure A.31. My account (registrar): name locked; a wrong current password is rejected (T53/T54). 30

Figure A.32. Forgot-password request form. 31

Figure A.33. Request received: the reference code is shown once (T55). 31

Figure A.34. Administrator inbox: requester, e-mail check against the profile, approve or reject with a reply (T57). 32

Figure A.35. Status page: approved, with the administrator's reply (T56). 32

Figure A.36. One-time temporary password, revealed only to the requester (T56). 33

Figure A.37. Signing in with the temporary password forces a password change (T56). 33

Figure A.38. student01 changes the id in the URL to another student: HTTP 403 (T07). 34

Figure A.39. student01 sends a grade-modification request directly: HTTP 403 (T08). 34

Figure A.40. lecturer01 modifies a grade in unassigned NW301: HTTP 403 (T11). 34

Figure A.41. registrar01 modifies a grade: HTTP 403 (T13). .35

Figure A.42. auditor01 modifies a student record: HTTP 403 (T15).....35

Figure A.43. admin01 (ADMIN only) opens a grade page: HTTP 403 (T17).....35

Figure A.44. Assigning AUDITOR to an administrator violates separation of duties: HTTP 403 (T30). 36

Figure A.45. Stored <script> payload is displayed as plain text and does not execute (T19). 36

Figure A.46. After logout the old session is rejected: HTTP 401 (T04).....37

Figure A.47. Security Auditor dashboard: audit volume, denials and failed logins. 37

Figure A.48. Security audit log with timestamp, actor, event, target, result and details (T14). 38

Figure A.49. Audit log filtered on rejected actions (T20). 38

Figure A.50. Integrity check: the SHA-256 hash chain is intact (T42).....39

Figure A.51. Read-only view of the access-control configuration (T14). 41

Figure A.52. users.password_hash contains only Argon2id hashes (T05). 41

Figure A.53. Automated security test results (pytest) (T01–T58). 41

LIST OF TABLES

Table 3.1. Threat model 3
Table 4.1. Access-control matrix 4
Table 7.1. Test results by category.....9
Table 8.1. Remaining limitations 10
Table 9.1. Individual contribution summary.....11
Table B.1. Complete security test results.....42
Table C.1. Security requirements, threats and verifying tests .48
Table C.2. Demo accounts (password of every account: <username>@Demo2026).....48

LIST OF ABBREVIATIONS

Abbreviation	Meaning
RBAC	Role-Based Access Control
IDOR	Insecure Direct Object Reference
SQLi	SQL Injection
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
CSP	Content Security Policy
DFD	Data-Flow Diagram
SoD	Separation of Duties
DBA	Database Administrator
MFA	Multi-Factor Authentication
TLS	Transport Layer Security
HSTS	HTTP Strict Transport Security
SHA-256	Secure Hash Algorithm, 256-bit output
OWASP	Open Worldwide Application Security Project
SR / TH / R	Security requirement / Threat / Remaining risk identifiers used in this report

ABSTRACT

This report documents the design, implementation and audit of access control for a small web-based University Records System that stores user accounts, student profiles, courses, enrollments, grades and a security audit log. A threat model with twelve threats and sixteen security requirements was derived first, followed by an access-control matrix for five roles (Student, Lecturer, Registrar, Security Auditor and System Administrator) based on default deny, least privilege and separation of duties. The system was implemented with Flask and PostgreSQL 16 and enforces authorization at three layers: an endpoint permission check, a record-level ownership or course-assignment check, and database privileges and triggers. Passwords are hashed with Argon2id, sessions are server-side, every state-changing request requires an anti-CSRF token, and sensitive events are written to an append-only, hash-chained audit log. Students can register themselves (STUDENT role only) and join classes of at most 30 students with one lecturer, and course totals are computed by the database from four weighted grade components. Every user can change their password, and forgotten passwords are reset through an administrator-approved workflow in which no one, not even the administrator, ever sees a stored password. An automated suite of 58 security tests, including the 20 mandatory tests, passed completely. The report closes with seven remaining risks and the improvements recommended for each.

Keywords: role-based access control, default deny, IDOR, Argon2id, session management, CSRF, SQL injection, XSS, audit logging, least privilege, PostgreSQL.

CHAPTER 1. INTRODUCTION

1.1 Background and Objectives

Broken access control is ranked first in the OWASP Top 10: a single missing check lets one user read or change another user's data, and in university records that means personal data and grades. The objective is to secure such a system end to end: identify the threats, define an explicit access-control policy, implement it on the server and in the database, verify it with repeatable tests, and review the remaining risks.

1.2 Scope and Group Organization

The scope covers authentication, sessions, roles, server-side and record-level authorization, the academic features, database security, validation, output encoding and audit logging. Primary responsibilities were Student A (Trieu Ngoc Van Anh): threat modeling and access-control design; Student B (Duong Tran Quoc Khanh): authentication, authorization and implementation; Student C (Nguyen Vo Thanh Tu): database security, testing and audit logging. All members reviewed the complete submission.

CHAPTER 2. SYSTEM OVERVIEW

The University Records System is a server-rendered web application written in Python 3.11 with Flask 3.1 and served by gunicorn. Data is stored in PostgreSQL 16. The application and the database run as two Docker containers defined in `docker-compose.yml`; both ports are published on 127.0.0.1 only. The seed data contains 11 demo accounts (including a disabled account and the multi-role teaching assistant `ta01`, Student and Lecturer), five classes (IS101, DB201, NW301, SE202 and the still unassigned AI305), six student profiles, eleven enrollments and six grade records.

Beyond the required operations, visitors can **register a student account**, students and lecturers use a **class catalog** to join a class or take an unassigned one, the registrar **creates classes** (one lecturer, at most 30 students) and arranges their

lecturer and students, and lecturers enter **four grade components** (10/20/20/50 %). Every user can change their password, and a **forgot-password** request is answered by an administrator.

The interface has a role-aware sidebar and a dashboard per role; permission lists are shown only to administrators and auditors. Hiding links is a convenience; every decision is taken on the server (screens in Appendix A).

CHAPTER 3. THREAT MODEL AND SECURITY REQUIREMENTS

3.1 Assets, Users and Trust Boundaries

Assets: A1 credentials, A2 session tokens, A3 personal data, A4 grades, A5 enrollments, A6 roles and policy, A7 audit log, A8 secrets. **Users:** the five roles (any may be malicious or compromised), anonymous visitors and the DBA. **Trust boundaries:** TB1 lies between the browser and the application (every request field, URL and cookie is untrusted). TB2 lies between the application and PostgreSQL (the application uses the restricted account `uni_app`). TB3 is the privileged maintenance channel (the Docker host and the database superuser), which lies outside the application (Figure 3.1).

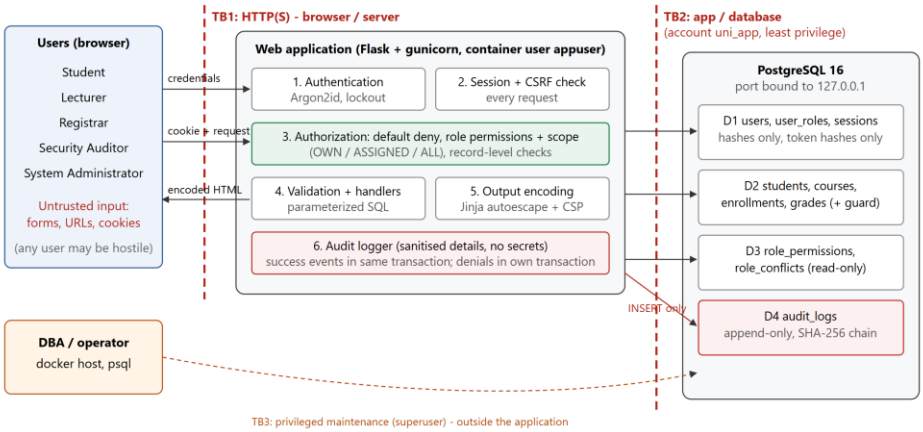


Figure 3.1. *System and data-flow diagram with trust boundaries TB1–TB3 and data stores D1–D4.*

3.2 Threat-Model Table

Likelihood (L) and impact (I) are rated High, Medium or Low before controls. Residual risks are in Chapter 8; the sixteen derived security requirements are traced to tests in Appendix C.

Table 3.1. *Threat model*

ID	Threat	Asset	Attack scenario	L	I	Mitigation / control
TH1	Account compromise	A1, A2	Password guessing or credential stuffing on /login; username enumeration via messages or timing.	H	H	Argon2id; one generic error; dummy hash for unknown users; lockout after 5 failures (15 min); disabled accounts rejected; LOGIN_FAILURE logged.
TH2	Session hijacking / fixation	A2	Stolen or planted session cookie; cookie replayed after logout.	M	H	256-bit random token renewed at login, stored only as SHA-256; HttpOnly, SameSite=Strict; 30-min idle / 8-h absolute expiry; revoked at logout and on disable.
TH3	Privilege escalation	A6	Forged requests to admin routes; administrator grants themself REGISTRAR; conflicting roles combined.	M	H	Default-deny decorator on every route; no self role change; role-conflict table checked in app and by trigger; policy tables read-only.
TH4	Insecure direct object reference	A3–A5	Student changes /students/1/grades to /students/2/grades.	H	H	Record-level OWN / ASSIGNED / ALL check in every handler; same 403 for foreign and missing ids; RECORD_ACCESS_DENIED logged.
TH5	SQL injection	A1–A8	' OR '1'='1' -- or UNION SELECT password_hash in login or search.	M	H	Bound parameters for every query; LIKE wildcards escaped; allow-list validation; least-privilege DB account; generic errors.
TH6	Cross-site scripting	A2, A3	Student stores <script> in the address field, which runs when staff view the profile.	M	H	Jinja2 auto-escaping (no |safe); CSP script-src 'self' with no inline script; HttpOnly cookie.
TH7	Unauthorized grade modification	A4	Student, registrar or unassigned lecturer posts /grades directly; teaching assistant grades their own enrollment.	H	H	grade.write only for LECTURER with scope ASSIGNED; self-grading blocked; DB trigger grades_guard repeats the checks; all attempts logged.
TH8	Audit-log tampering	A7	Insider deletes or edits log entries to hide an action.	M	H	App account may only INSERT/SELECT; triggers block UPDATE/DELETE/TRUNCATE; SHA-256 hash chain with verification by the auditor.
TH9	Leakage of credentials or records	A1, A3, A8	Secrets in Git, stack traces or SQL in error pages, tokens in logs.	M	H	.env git-ignored; debug off and generic error pages; audit details sanitized; only hashes stored; ports bound to localhost.
TH10	Cross-site request forgery	A4, A6	A malicious page makes a lecturer's browser submit a grade.	M	H	Per-session synchronizer token on every POST; SameSite=Strict; CSRF_REJECTED logged.
TH11	Abuse of self-service (registration, classes)	A3–A6	Sign-up with a forged role or by claiming another student's record; over-filling a class; a 2nd lecturer taking a class.	M	H	Sign-up always STUDENT with a NEW profile; class capacity ≤ 30 and one lecturer enforced in app and DB (trigger, UNIQUE); self-service limited to oneself; all actions logged.
TH12	Account takeover via password reset	A1, A2	Forged reset request; an administrator reading users' passwords.	M	H	Uniform response; secret code; admin check and approval; one-time 24 h temp password shown only to the requester; forced change.

CHAPTER 4. ACCESS-CONTROL POLICY

4.1 Access-Control Matrix

In Table 4.1, Allow means the operation is permitted, Own means only the user's own record, Assigned means only courses assigned to the lecturer, Deny means the operation is explicitly prohibited, and N/A means the role has no such object. The same policy is stored in the database table `role_permissions`, and the auditor can view it at `/audit/policy`.

Table 4.1. *Access-control matrix*

Resource and operation	Student	Lecturer	Registrar	Auditor	Administrator
View own profile, enrollments and grades	Own	N/A	N/A	N/A	N/A
Update own contact information	Own	Deny	via record update	Deny	Deny
View another student's profile	Deny	Assigned course only	Allow	Deny	Deny
Search, create and update student records	Deny	Deny	Allow	Deny	Deny
View courses and course roster	Deny	Assigned only	Allow	Deny	Deny
Create / drop enrollment	Deny	Deny	Allow	Deny	Deny
Create class (capacity ≤ 30)	Deny	Deny	Allow	Deny	Deny
Assign / replace / remove class lecturer (max. 1)	Deny	Deny	Allow	Deny	Deny
Join / leave a class (free seat; leave only if ungraded)	Own	Deny	via enrollment	Deny	Deny
Take an unassigned class as its lecturer	Deny	Own, only if no lecturer	Deny	Deny	Deny
View another student's grade	Deny	Assigned course only	Allow	Deny	Deny
Create / modify grade components	Deny	Assigned course only, not own enrollment	Deny	Deny	Deny
Change own password (current password required)	Own	Own	Own	Own	Own
Change own display name	Own	Own	Deny	Deny	Deny
Answer forgotten-password requests (not own)	Deny	Deny	Deny	Deny	Allow
View any stored password	Deny	Deny	Deny	Deny	Deny (not stored)
Read audit log, verify integrity, read access policy	Deny	Deny	Deny	Allow (read-only)	Deny
Create / disable / enable / unlock user	Deny	Deny	Deny	Deny	Allow (not disable self)
Assign / remove roles	Deny	Deny	Deny	Deny	Allow (not own roles, no conflicting pair, keep one admin)
Modify permissions and role definitions	Deny	Deny	Deny	Deny	Deny (DBA migration only)

Resource and operation	Student	Lecturer	Registrar	Auditor	Administrator
Delete students, grades, users, audit entries	Deny	Deny	Deny	Deny	Deny

Every role may log in, log out and open its dashboard. **Anonymous visitors** may only sign in or register; registration always creates a STUDENT account with a new student profile.

4.2 Policy Explanation

Default deny. An operation is allowed only if a row in `role_permissions` grants it. Every route must declare its rule (`@public`, `@login_required`, `@require_permission`); `verify_all_endpoints_declared()` stops the application at start-up otherwise. Requests without a session get 401; without the permission 403, logged, with no data changed.

Least privilege. Each role holds only the permissions of its duties: the Registrar has `grade.read` but not `grade.write`, and the Administrator has no academic permission. Below the application, `uni_app` can neither delete academic data nor modify the policy (Chapter 6).

Separation of duties and role conflicts. The table `role_conflicts` forbids three combinations: ADMIN+AUDITOR (whoever manages accounts must not audit those actions), LECTURER+REGISTRAR (enrolling and grading must be done by different people) and STUDENT+REGISTRAR (a registrar must not maintain their own record). Conflicts are checked when a role is assigned and again by the trigger `user_roles_conflict_guard`. An administrator cannot change their own roles, so separate academic access for an administrator requires a second administrator.

Multiple roles. The effective permissions of a user are the union of the (permission, scope) pairs of all of the user's roles. The union never widens a scope. For example, `ta01` (Student+Lecturer) can read their own grades (OWN) and grade SE202 (ASSIGNED), but cannot open the DB201 roster or grade their own SE202 enrollment (tests T27 and T28).

Self-service and classes. Registration ignores any role field and never links an existing record, so it cannot be used to read another student's grades. Self-service class actions act only on the user's own enrollment or teaching assignment; arranging other people remains a registrar duty (`course.manage`, `enrollment.manage`). A lecturer may take only a class without a lecturer and in which they are not enrolled. Renaming is denied to holders of ADMIN, AUDITOR or REGISTRAR (deny overrides); the administrator answers reset requests but never learns the new password.

Ownership and course-assignment checks. Handlers load the object and call `can_access_student()` or `can_access_course()`: OWN requires `students.user_id` = session user, ASSIGNED a `course_lecturers` row. For a grade the course is taken from the stored enrollment, never from the client.

CHAPTER 5. SECURITY ARCHITECTURE AND IMPLEMENTATION

Every request passes the same pipeline (Figure 3.1): session check, route access rule, CSRF token, input validation and record scope, parameterized SQL in one transaction, then auto-escaped output with security headers (CSP, X-Frame-Options DENY, nosniff, no-referrer, no-store).

5.1 Authentication and Session Handling

Passwords are hashed with Argon2id (64 MiB, 3 iterations, 4 lanes); a CHECK constraint rejects any other value. Wrong passwords, unknown, disabled and locked accounts all return "Invalid username or password.", and unknown usernames still cost one hash verification. Five failures lock an account for 15 minutes. Each login issues a new random 256-bit token. Only its SHA-256 hash is stored in the `sessions` table, together with the CSRF token and the expiry. Logout revokes the session on the server, and disabling an account revokes all of that account's sessions (`security.py`, `routes/auth.py`). Registration and password change apply the same

policy (12–128 characters, confirmation, not containing the username); a change needs the current password (failures count towards lockout) and revokes other sessions. For a reset (`routes/account.py`) the requester gets a 96-bit code (stored as SHA-256); after approval a temporary password is generated on reveal, shown once, valid 24 h, and forces a password change.

5.2 Authorization

`source/app/authz.py` loads the user's roles and the scoped permissions from the database on every request, so a role change takes effect immediately. The decorator checks the permission. The handlers in `routes/students.py` and `routes/courses.py` then check the scope of the specific record. Grade writes additionally reject self-grading and inactive enrollments. The administration handlers enforce the self-change, role-conflict and last-administrator rules.

5.3 Grading and Class Management

A grade row holds four components; the course total `score` is a generated column (0.1·P1 + 0.2·P2 + 0.2·Mid + 0.5·Final), so it cannot be written by any client and stays empty until all components exist. The student page shows every component and a credit-weighted overall average. Class handlers check capacity and the single-lecturer rule before writing; the database repeats both.

5.4 Input Validation and Output Encoding

`validators.py` applies allow-list rules (formats, lengths, no control characters, scores 0–10 with two decimals); bodies over 64 KB are rejected. Templates are auto-escaped and use only inline SVG, one stylesheet and one script (`static/app.js`, usability only), so the CSP `script-src 'self'; style-src 'self'` holds. Toast messages come from a fixed server-side list (`?notice=grade_saved`), so no request text is reflected.

5.5 Security Audit Logging

`audit.py` records the eight required event types plus registration, lockout, CSRF, class and record events, each with timestamp, actor, event type, target, result and JSON details (e.g. old and new grade components); secret keys are stripped. Successful changes are logged in the same transaction, rejections in a separate one, so they are recorded although nothing changes.

5.6 Error Handling and Secrets

Debug mode is off; generic pages for 400–500 show no stack trace, SQL or configuration. A database guard violation (42501) becomes 403 (DB_GUARD_DENIED), a constraint violation (class full, second lecturer) 400. Secrets live only in the git-ignored `.env`; the container runs as an unprivileged user.

CHAPTER 6. DATABASE-SECURITY CONTROLS

- **Separate accounts.** The DBA account `uni_dba` is used only for initialization. The application connects as `uni_app` (password from the environment, created by `database/init/00-create-roles.sh`): no superuser, CREATEROLE, CREATEDB or BYPASSRLS, owns no table; CONNECT is revoked from PUBLIC.
- **Table and column grants.** No DELETE on users, students, enrollments or grades; UPDATE only on named columns (the four grade components, never the total; no username or class-code change); policy tables SELECT-only; `audit_logs` INSERT and SELECT only; `password_reset_requests` has no password column, no DELETE and only its workflow columns can be updated.
- **Integrity triggers.** `grades_guard` rejects any grade written by someone who is not an active, assigned lecturer, and rejects self-grading, even if the application layer were bypassed. `user_roles_conflict_guard` enforces separation of duties. `enrollments_capacity_guard` locks the class row and rejects a student beyond its capacity ($\leq$ 30); `UNIQUE (course_id)` allows one lecturer and `course_lecturers_guard` only active LECTURERs. CHECK constraints protect formats and score ranges.

- **Append-only, tamper-evident audit log.** A BEFORE INSERT trigger assigns the id, a server timestamp, `prev_hash` and `entry_hash` = SHA-256(prev_hash | fields) under an advisory lock. UPDATE, DELETE and TRUNCATE are blocked for every user, including the DBA. `audit_verify_chain()` reports the first altered entry.
- **Credential protection.** Only Argon2id and SHA-256 token hashes are stored; database logins use SCRAM-SHA-256; the port is bound to 127.0.0.1.

CHAPTER 7. SECURITY TESTING

7.1 Test Plan

The suite in `tests/` is automated and repeatable (pytest + requests against the running system, plus database checks). Each test declares ID, preconditions, role, action and expected result; the run records the actual result, PASS/FAIL and an evidence trace in `evidence/http/`. T01–T20 are mandatory; T21–T58 are original tests covering every additional control and feature.

7.2 Results

The final run on 2026-11-10 19:48:40 passed **58 of 58** tests (Table 7.1); all 20 mandatory tests T01–T20 passed. The complete table with preconditions, actions, actual results and evidence for every test is given in Appendix B.

Table 7.1. *Test results by category*

Category	Tests	Pass	Fail	Test IDs
Authentication	16	16	0	T01, T02, T03, T04, T05, T21, T22, T23, T24, T25, T45, T46, T53, T55, T56, T58
Authorization	29	29	0	T06, T07, T08, T09, T10, T11, T12, T13, T14, T15, T16, T17, T26, T27, T28, T29, T30, T31, T32, T33, T34, T47, T48, T49, T50, T51, T52, T54, T57
Injection / validation	6	6	0	T18, T19, T35, T36, T37, T38
Audit log	5	5	0	T20, T39, T40, T41, T42
Database security	2	2	0	T43, T44
Total	**58**	**58**	**0**	T01–T20 mandatory, T21–T58 original

Defects were found only in three tests (T24, T35, T42) and corrected.

CHAPTER 8. REMAINING RISKS AND IMPROVEMENTS

The final review checked the suggested areas against the actual code and configuration; Table 8.1 lists the remaining limitations; role explosion is low because permissions use scopes.

Table 8.1. *Remaining limitations*

#	Limitation (verified)	L	I	Risk	Recommended improvement	Implemented
R1	No multi-factor authentication.	M	H	High	TOTP/WebAuthn for staff roles.	No
R2	Reset depends on a manual identity check; admin-created and demo passwords are not force-changed.	M	M	Medium	Verified e-mail/SMS reset; force change of initial passwords.	Partly (change + approved one-time reset)
R3	HTTP only on localhost; Secure cookie and HSTS inactive.	L	H	Medium	TLS proxy, COOKIE_SECURE=true, HSTS.	Partly (config switch)
R4	No encryption at rest; no backups.	L	H	Medium	Encrypted volume and backups, restore tests.	No
R5	A DB superuser can disable triggers and rebuild the hash chain.	L	H	Medium	Anchor chain heads in WORM storage / SIEM.	Partly (append-only + chain)
R6	Lockout can be abused; no per-IP rate limit.	M	L	Low	Per-IP throttling, CAPTCHA.	Partly (admin unlock)
R7	No break-glass access; no CSRF token on login.	L	M	Low	Two-admin emergency role; login CSRF token.	No
R8	Sign-up has no e-mail check or CAPTCHA; lecturers take open classes without approval.	M	L	Low	E-mail verification, throttling, approval of teaching.	Partly (STUDENT only, logged)

CHAPTER 9. CONCLUSION

The group derived twelve threats and a default-deny, least-privilege policy with separation of duties, and implemented it at three layers: endpoint decorator, record-level scope check, and database privileges and triggers. All 58 automated tests passed, including the 20 mandatory tests. The main open risks are MFA, an automated reset channel, TLS, encryption at rest and external protection of the audit log.

INDIVIDUAL CONTRIBUTION SUMMARY

Each member's full statement (tasks completed, repository commits, technical decisions, problems solved, approximate hours and the contributions of the other members) is submitted separately in `contribution/member-a.pdf`, `member-b.pdf` and `member-c.pdf`. All members reviewed the complete submission.

Table 9.1. *Individual contribution summary*

Member	Primary responsibility	Summary of contribution
Student A Trieu Ngoc Van Anh	Threat modeling and access-control design	<to be completed by the member>
Student B Duong Tran Quoc Khanh	Authentication, authorization and implementation	<to be completed by the member>
Student C Nguyen Vo Thanh Tu	Database security, testing and audit logging	<to be completed by the member>

APPENDIX A. SYSTEM DEMONSTRATION

This appendix walks through the implemented system screen by screen, grouped by role. Every screenshot was captured automatically from the running application by `tests/capture_screenshots.py` (seed data, 1440 × 900 viewport). The figure captions name the test case that the screen illustrates, so each image can be traced to Appendix B.

A.1 Interface Design

The interface, branded UniRecords, uses one layout for all roles. A dark sidebar lists only the sections that the user's permissions allow, grouped as Overview, My studies, Academic, Administration and Security. A top bar shows a breadcrumb, the signed-in user with role badges and a logout button. Each page starts with a header and summary tiles, followed by cards holding tables or forms. Status is colour-coded consistently: grades by band (excellent, good, pass, fail), enrollments and accounts by badges, and audit results as Success, Denied or Failure.

Interaction is enhanced by the external script `static/app.js`: smooth page transitions, entrance animations, count-up statistics, a loading state on buttons, confirmation dialogs before destructive actions (drop, disable) and auto-hiding confirmation toasts. The script carries no security logic. Animations are disabled for users who prefer reduced motion, and the layout adapts to mobile screens with a slide-in menu. Because the icons are inline SVG and all styles and scripts are served from the application itself, the strict Content Security Policy remains unchanged.

A.2 Authentication

The sign-in page (Figure A.1) combines a product panel with the login form. The password field can be revealed with the eye button, and a link leads new students to registration. Whatever the reason for a failure (wrong password, unknown user, disabled or locked account, or an injection payload), the page shows the same message, as Figures A.2–A.4 show.

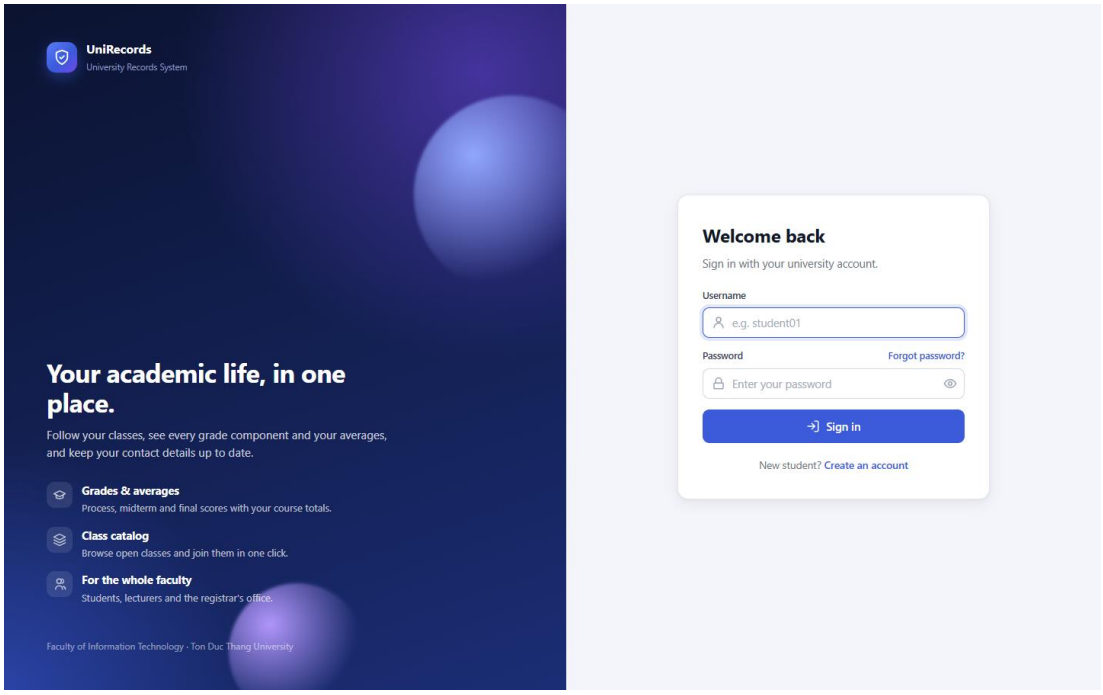


Figure A.1. *Sign-in page with the product overview and the login form.*

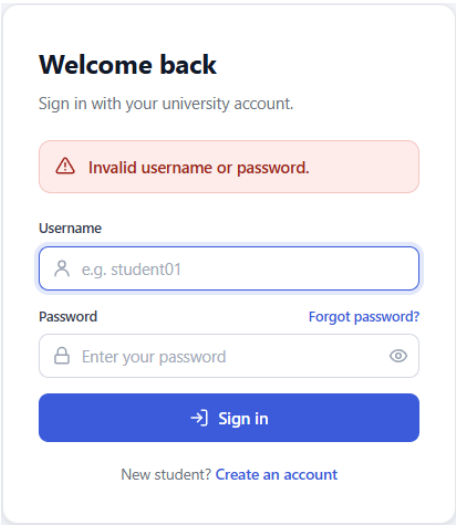


Figure A.2. *Wrong password: one generic error message (HTTP 401) (T02).*

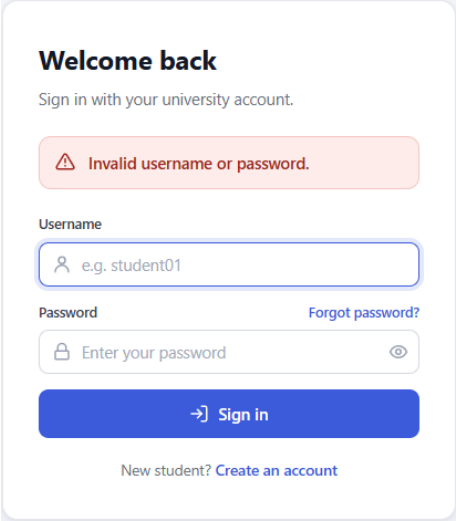


Figure A.3. *Disabled account with the correct password is rejected (T03).*

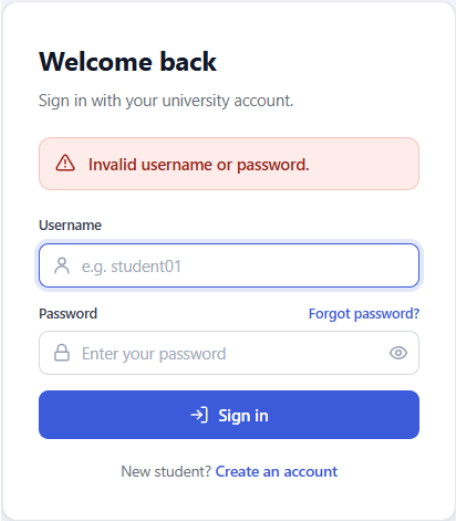


Figure A.4. *SQL-injection payload in the login form: generic failure, no database error (T18).*

A.3 Self-Registration and Class Catalog

A visitor creates a student account with name, username, e-mail and a confirmed password. Invalid input is rejected on the server with all messages at once. A successful sign-up returns to the login page with a confirmation; the new student receives a fresh student code and can immediately join an open class from the catalog, whose cards show lecturer, credits and seat usage.

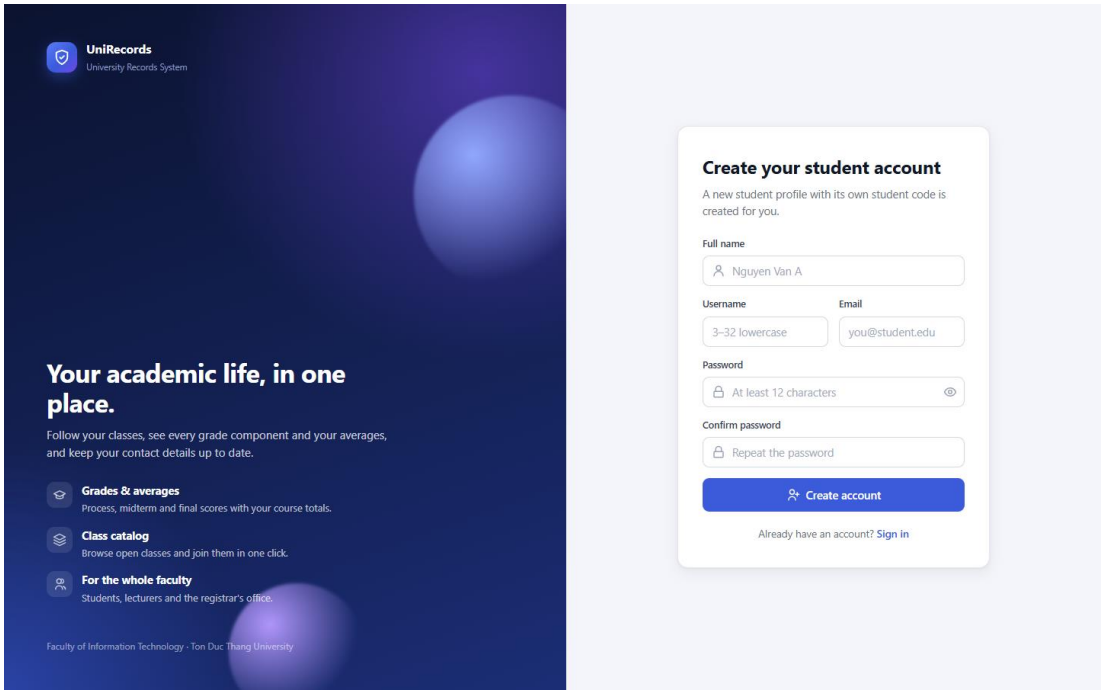


Figure A.5. *Student self-registration form.*

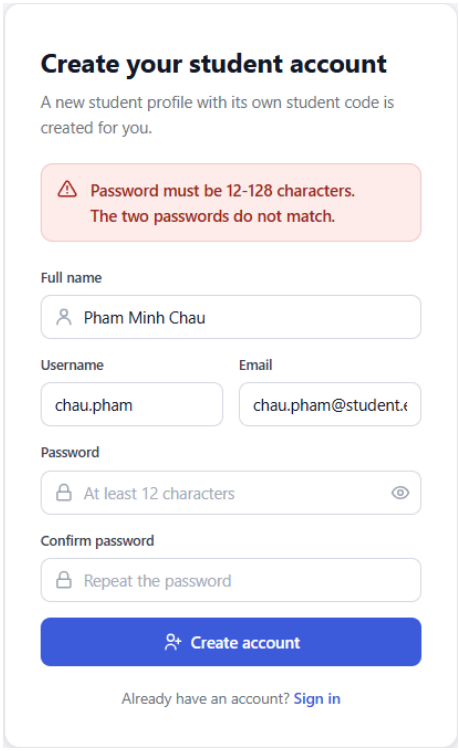


Figure A.6. *Registration rejected by server-side validation (T46).*

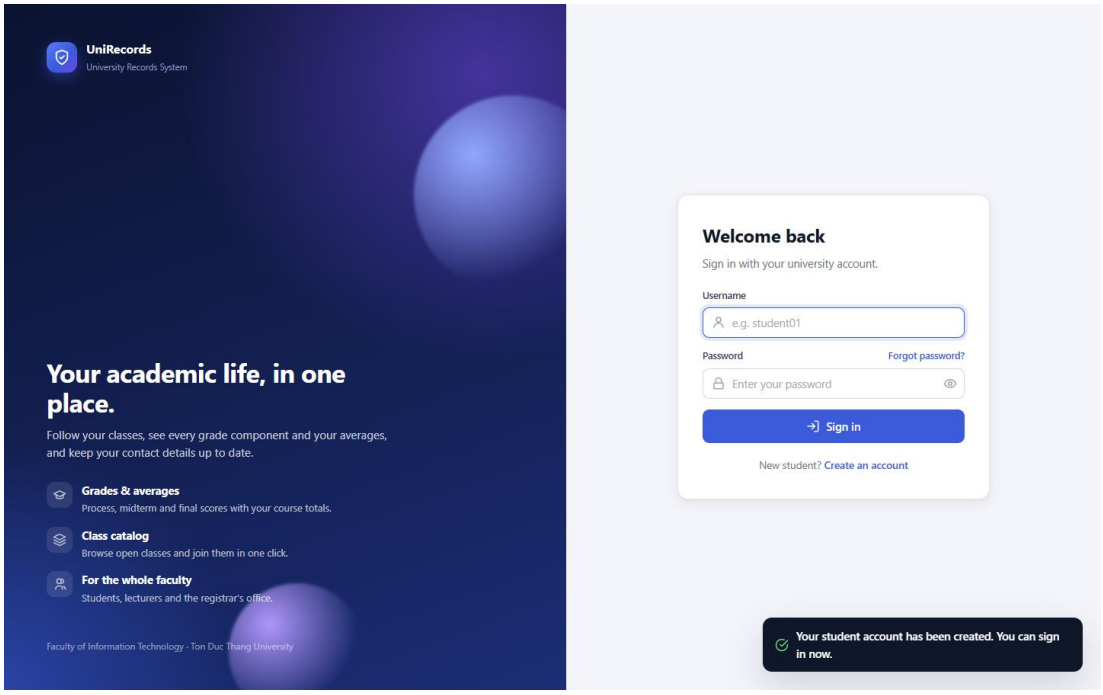


Figure A.7. *Account created with the STUDENT role only; the user can sign in (T45).*

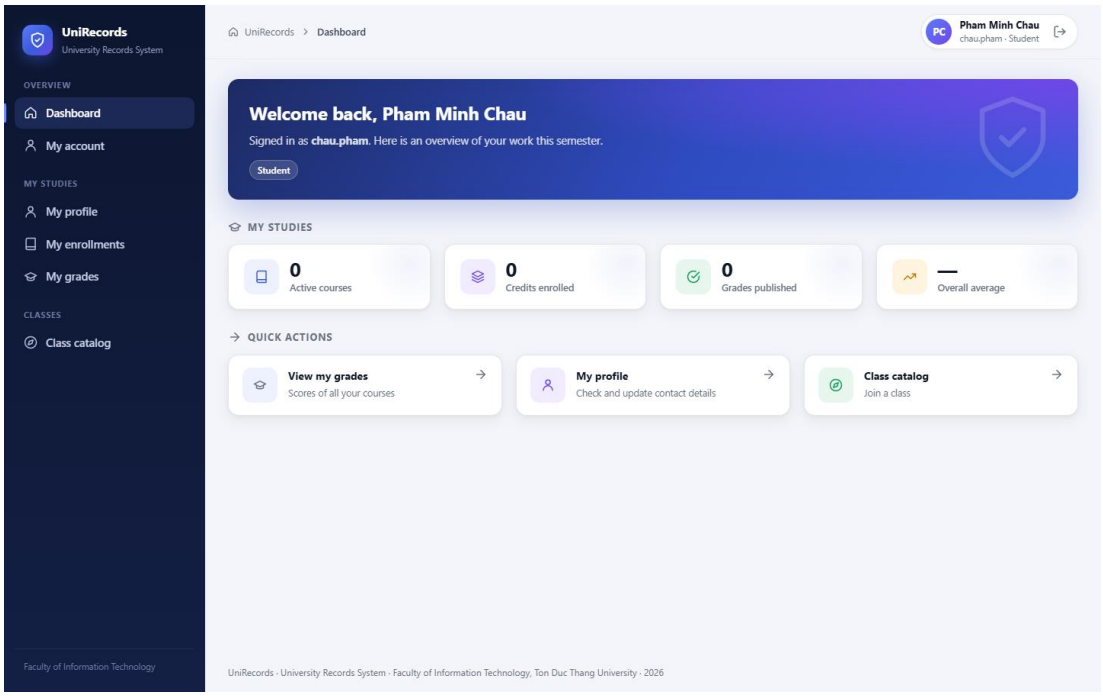


Figure A.8. *Dashboard of the newly registered student (new student code).*

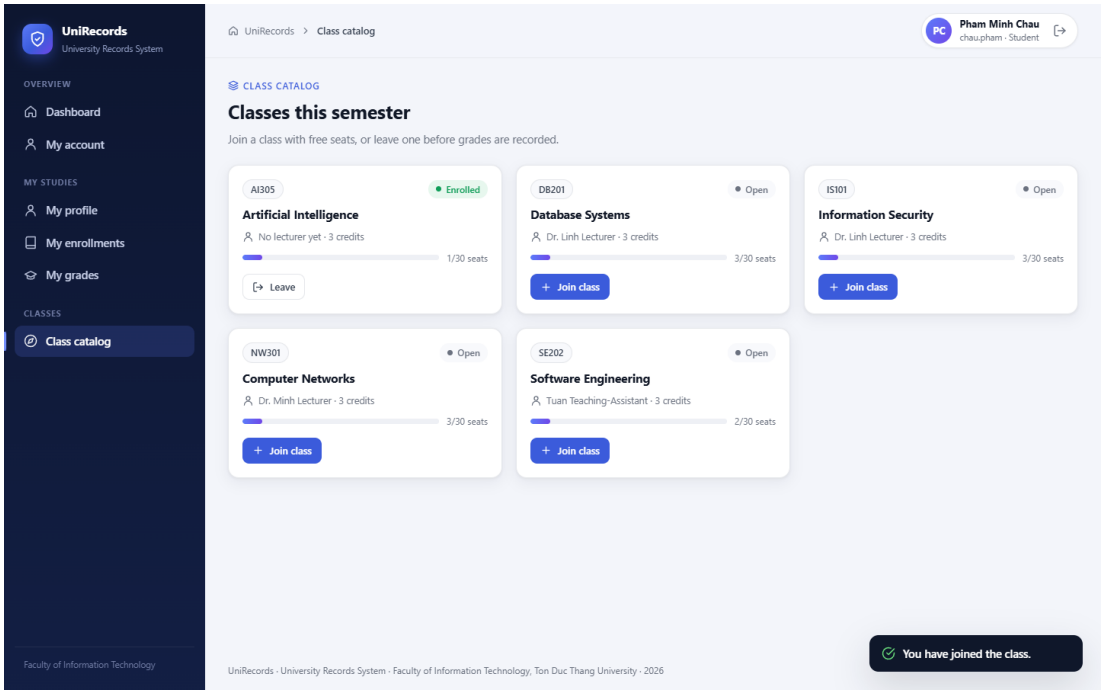


Figure A.9. *Class catalog: the new student joins AI305 (seat counter updated) (T51).*

A.4 Student Portal

A student's dashboard summarises their own studies (active courses, credits, published grades and the average) and offers quick actions. The profile page shows the official record next to a self-service form limited to email, phone and address. The grade page lists the four components of every course, the colour-coded total and the credit-weighted overall average. The catalog shows enrolled, open and full classes; a class cannot be left once grades exist.

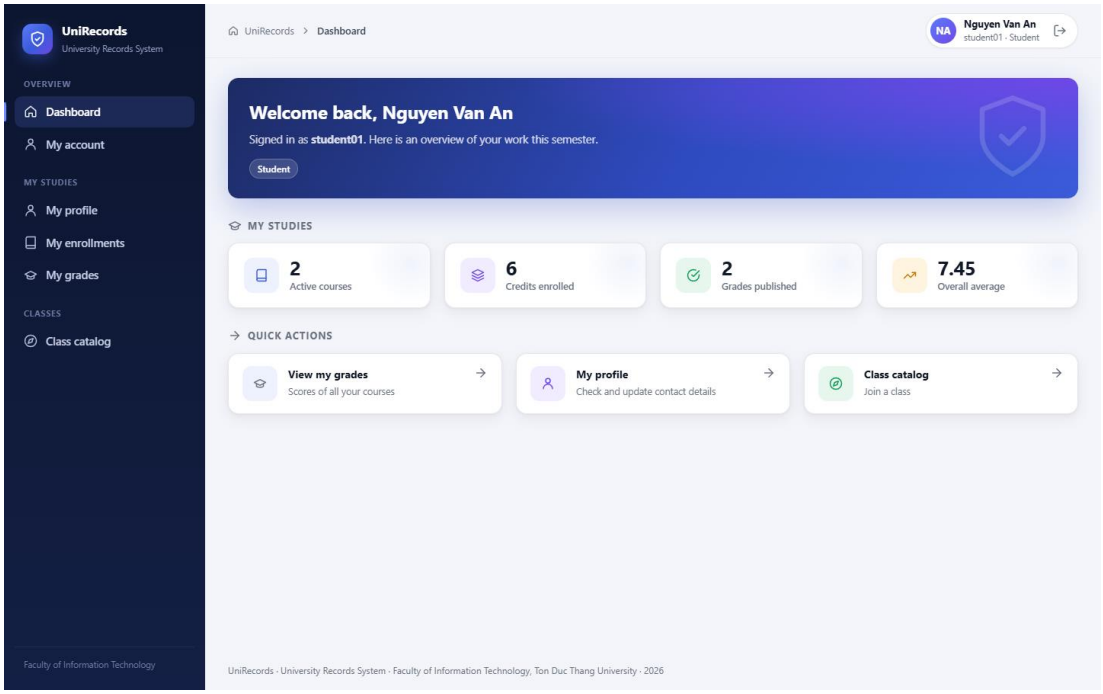


Figure A.10. *Student dashboard: study statistics and quick actions (T01).*

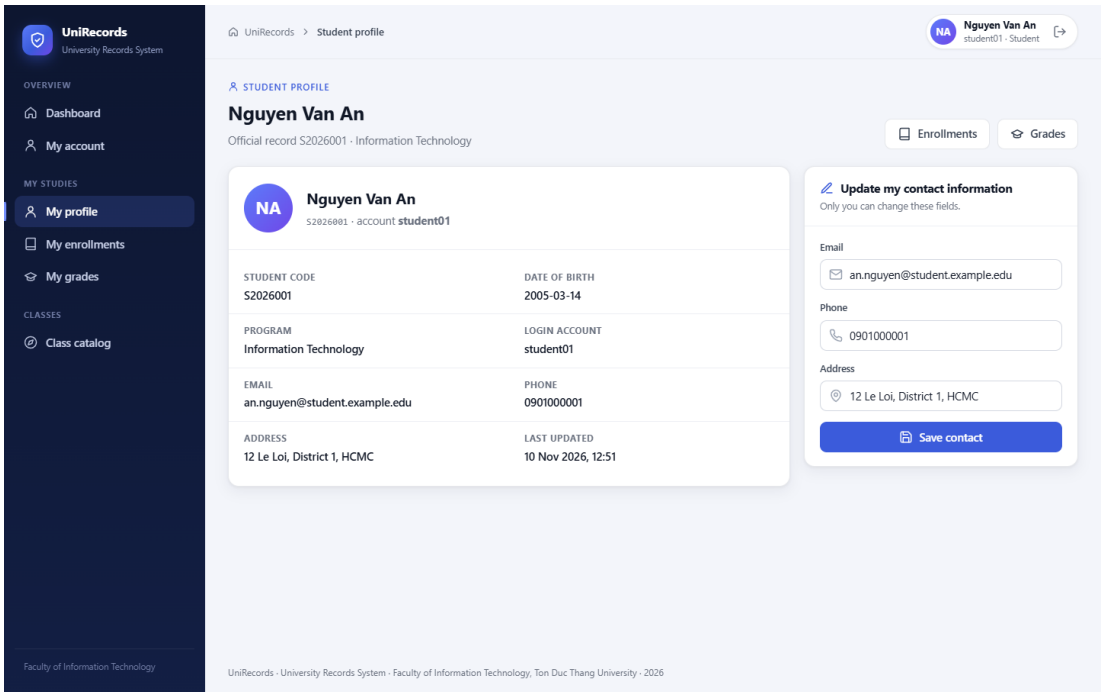


Figure A.11. *Student profile with the self-service contact form.*

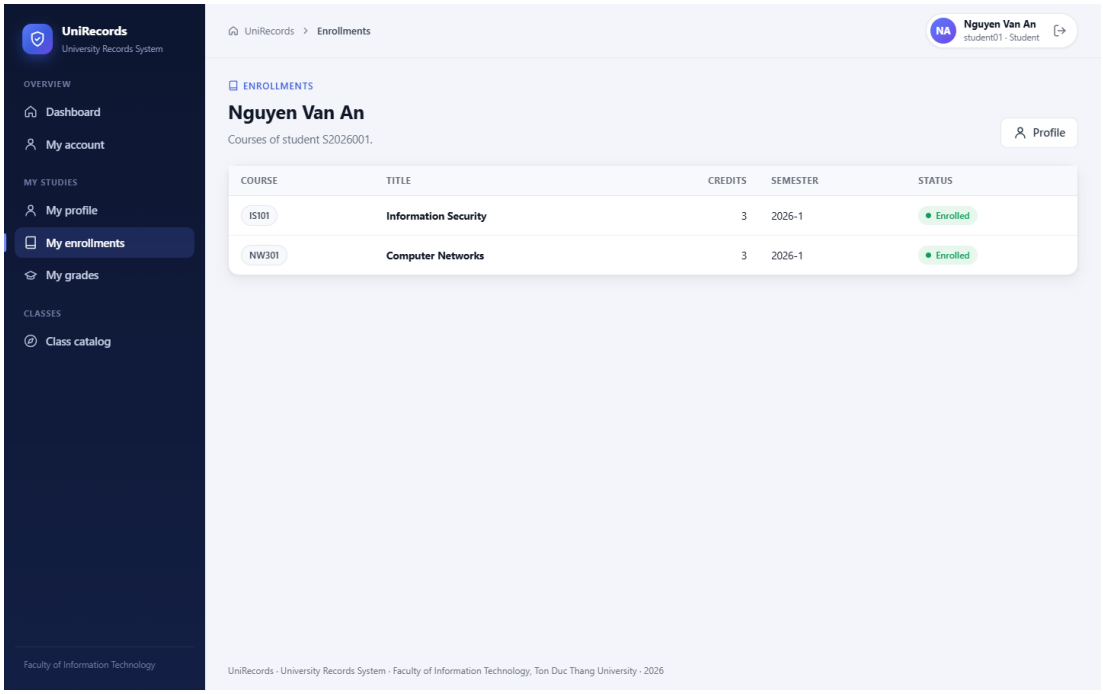


Figure A.12. *Student's own enrollments.*

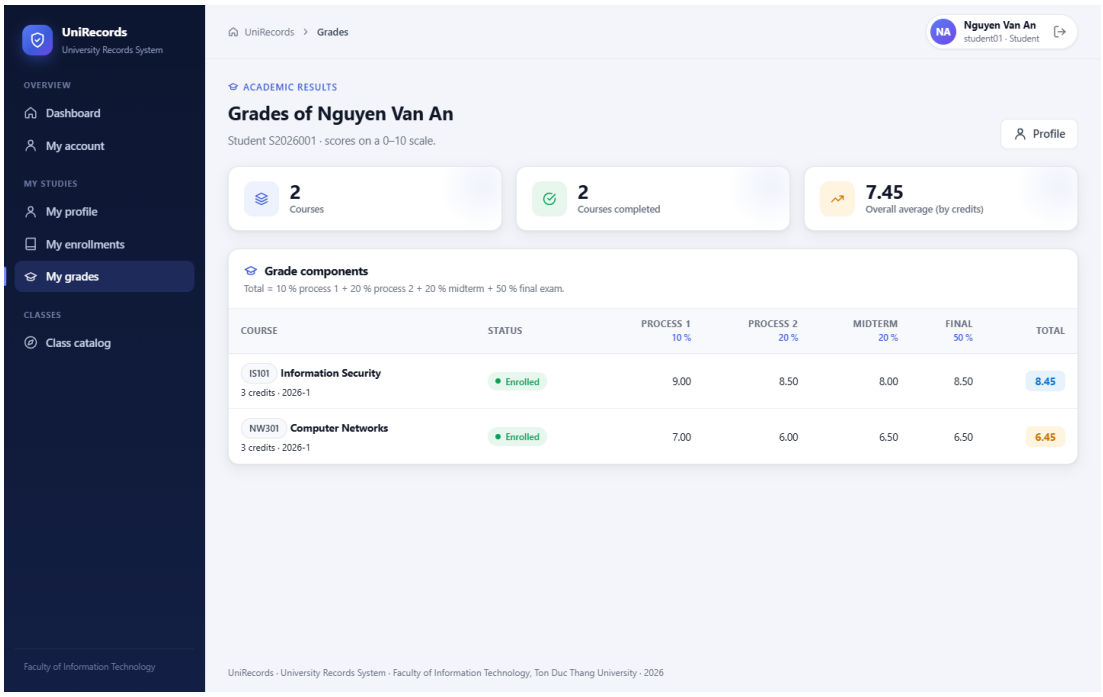


Figure A.13. *Own grades: four components, course totals and the credit-weighted overall average (T06).*

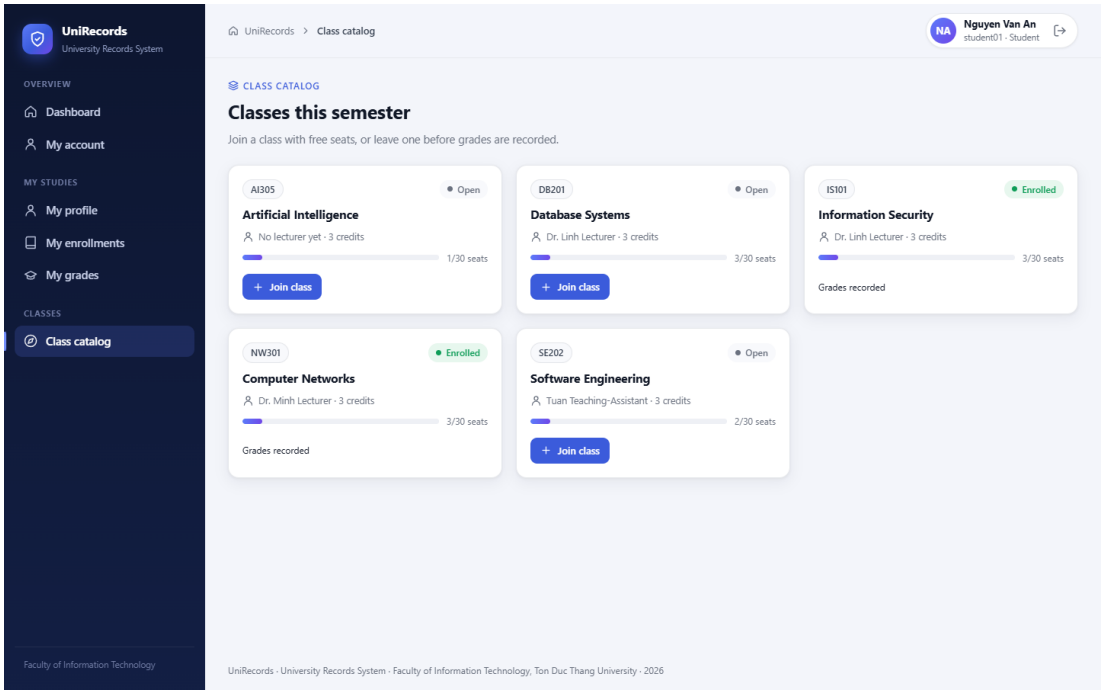


Figure A.14. *Class catalog seen by student01 (enrolled, open and full classes).*

A.5 Lecturer Workspace

Lecturers see only their assigned classes. The roster has one input per grade component with its weight and the computed total. After saving, a toast confirms the change, which is audited with old and new values. For the multi-role account `ta01`, the own SE202 enrollment shows "Own enrollment" instead of inputs; the server and the database trigger reject such a request anyway (T27). In the catalog a lecturer can take a class that has no lecturer.

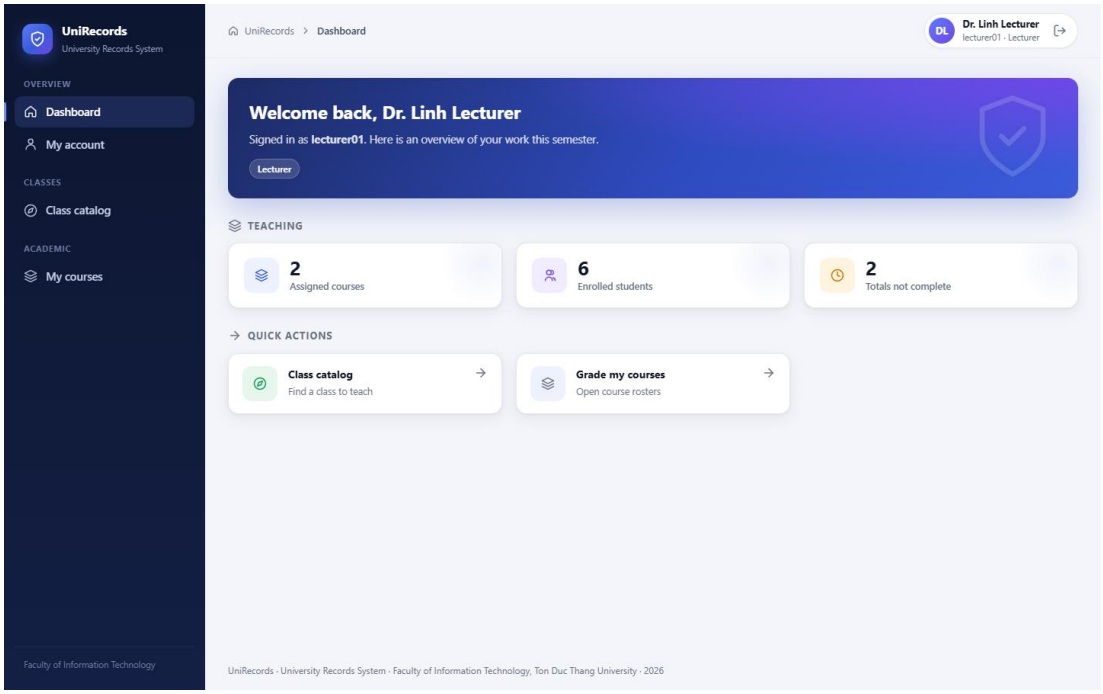


Figure A.15. *Lecturer dashboard: assigned classes, students and incomplete totals.*

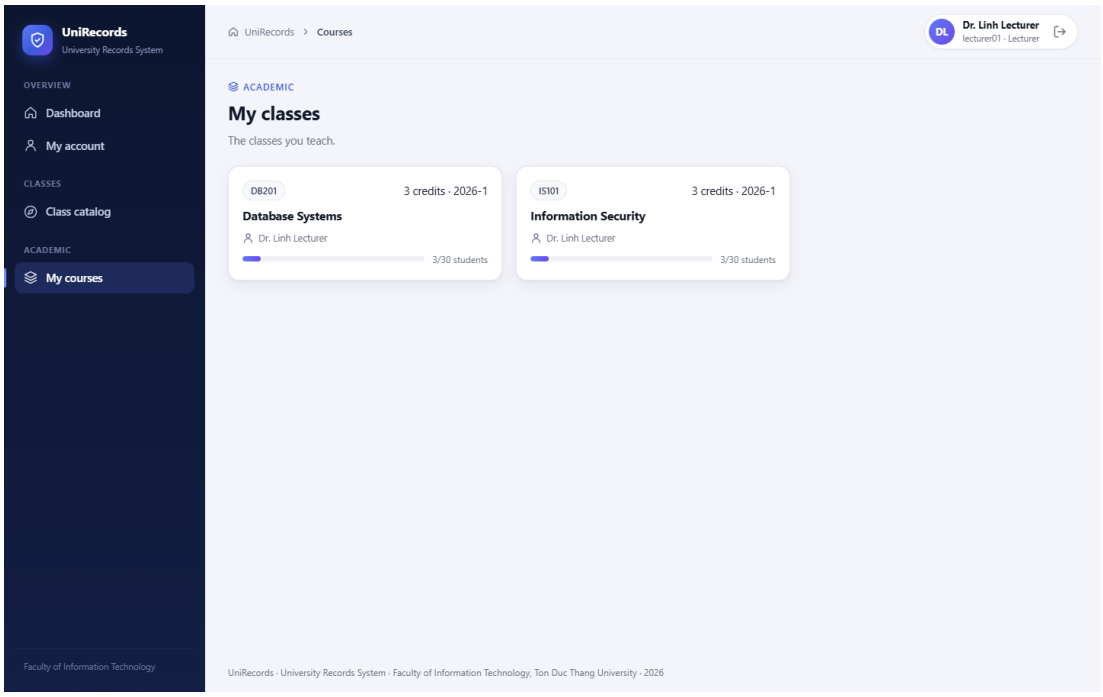


Figure A.16. *Lecturer sees only the classes assigned to them.*

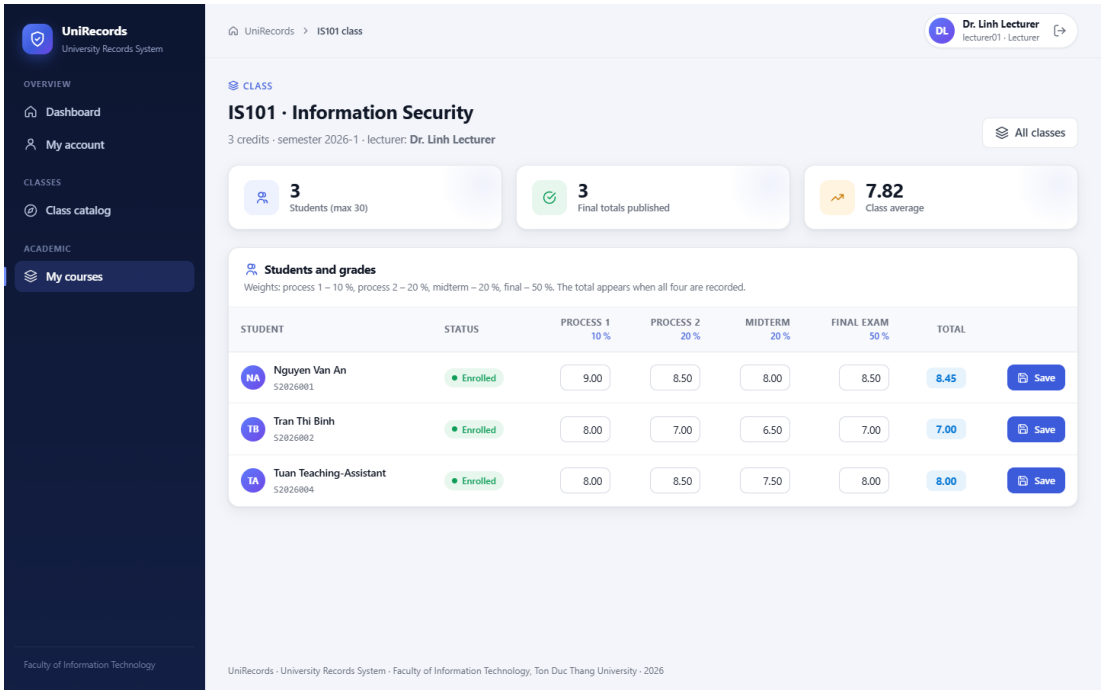


Figure A.17. *Roster of IS101 with the four grade components and totals (T09).*

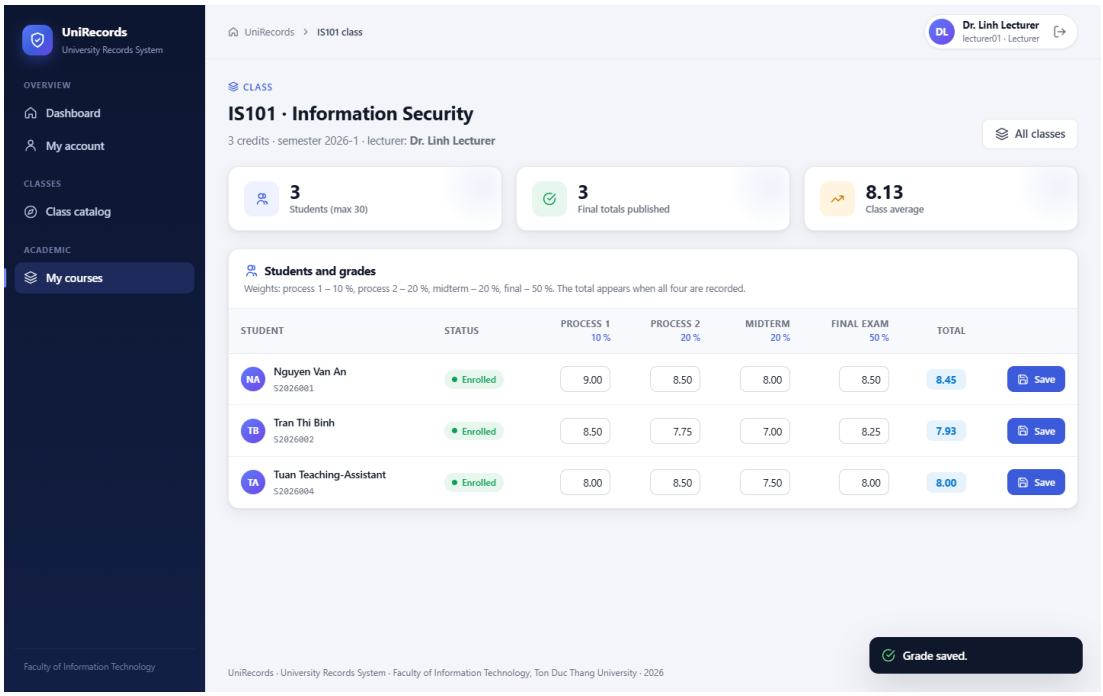


Figure A.18. *Components saved for S2026002; the database computes the total 8.00 (confirmation toast) (T10).*

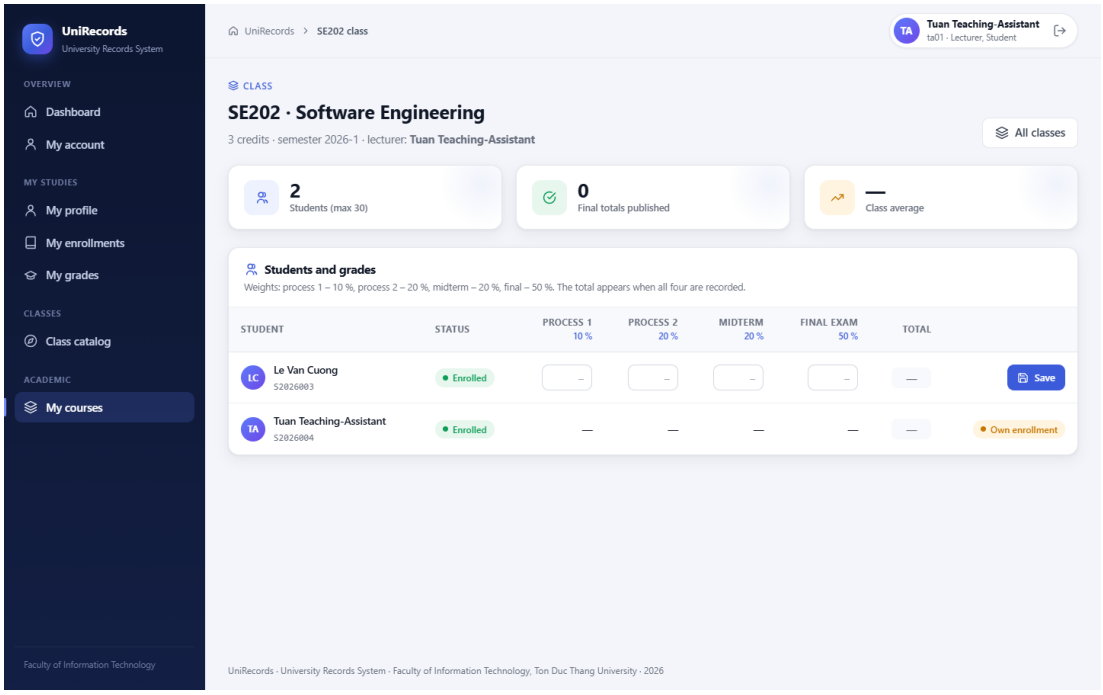


Figure A.19. *Multi-role ta01 teaches SE202 but cannot grade their own enrollment (T27).*

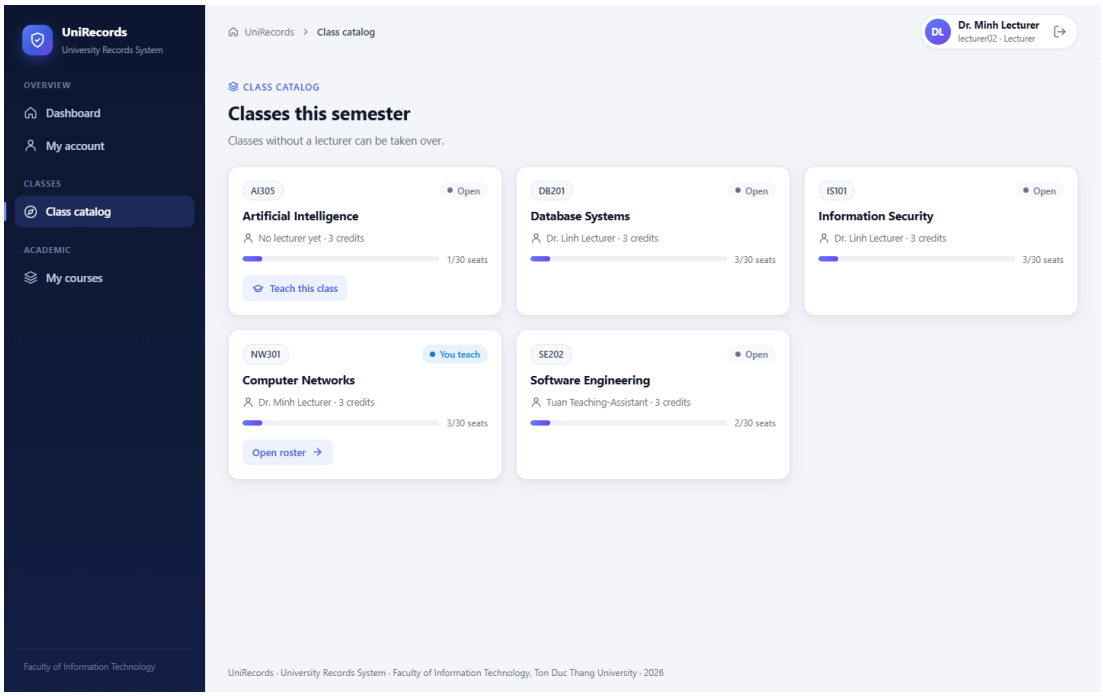


Figure A.20. *Lecturer catalog: an unassigned class can be taken over (T52).*

A.6 Registrar Workspace

The registrar works with all student records (searchable list, creation form) and all classes. A new class gets a code, title, credits, semester, a capacity of at most

30 and optionally its lecturer. The class page lets the registrar replace or remove the single lecturer, add students by code while seats remain, and remove students. The registrar can view grades but has no grade input anywhere.

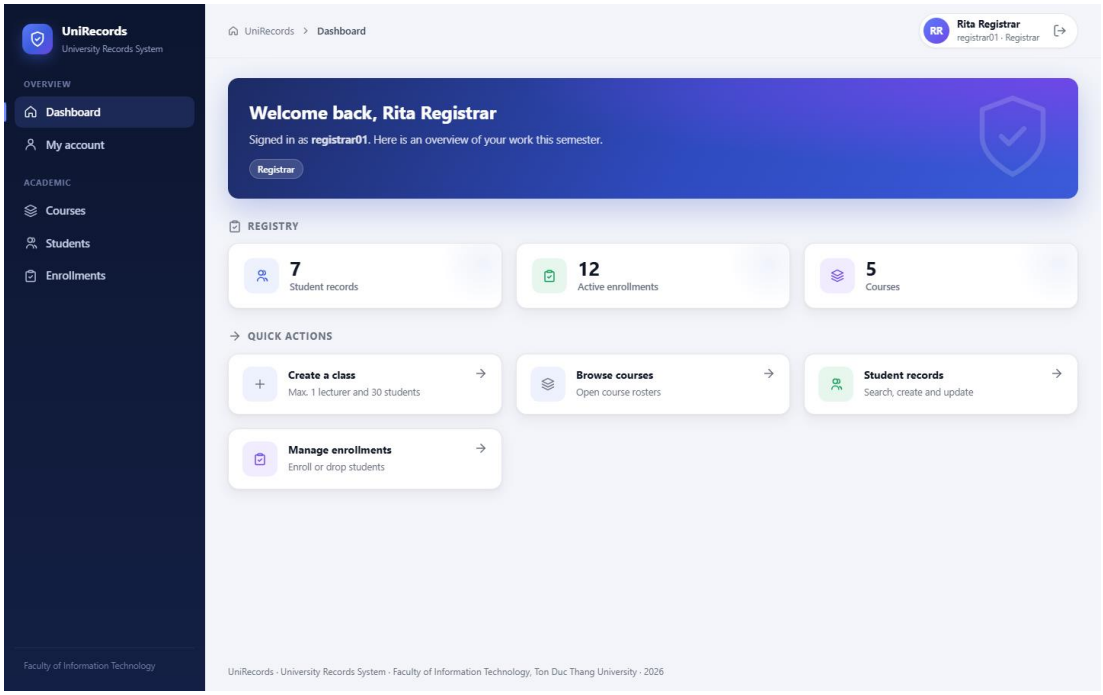


Figure A.21. *Registrar dashboard: records, enrollments and classes.*

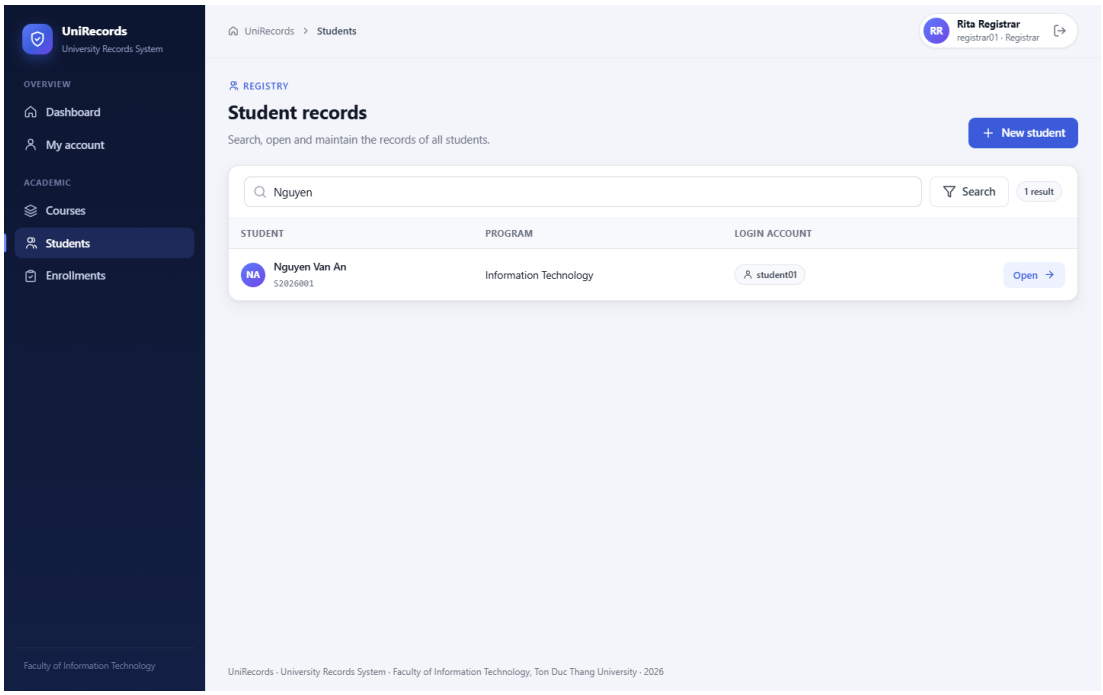


Figure A.22. *Student search (the search text is a bound SQL parameter).*

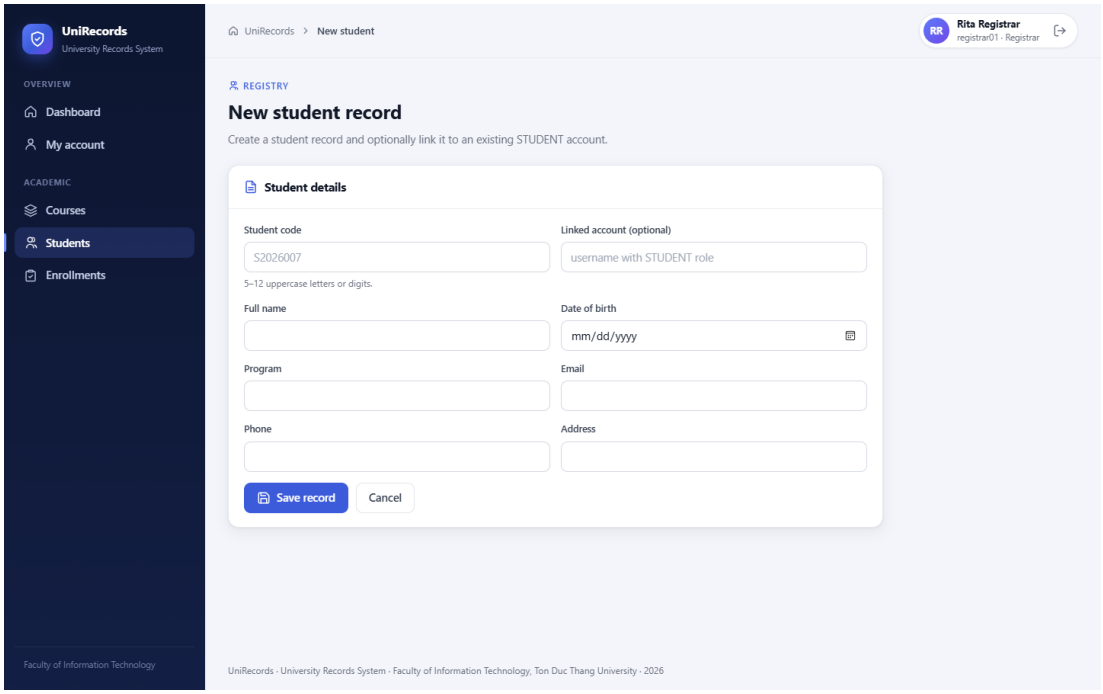


Figure A.23. *Form for creating a student record.*

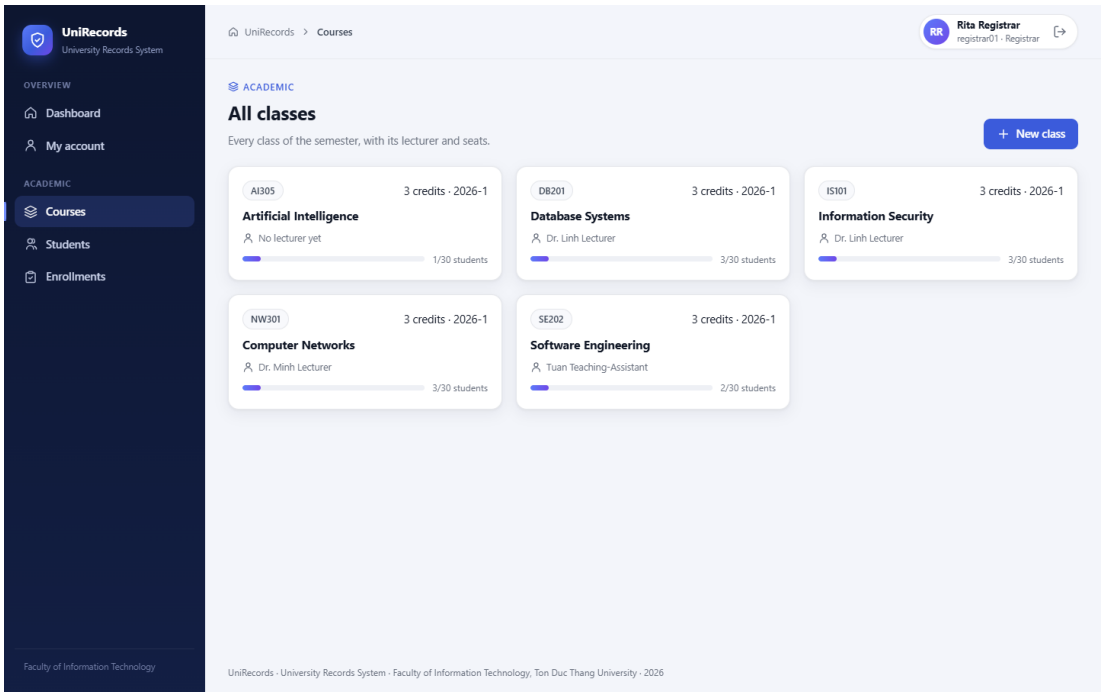


Figure A.24. *All classes with lecturer and seat usage.*

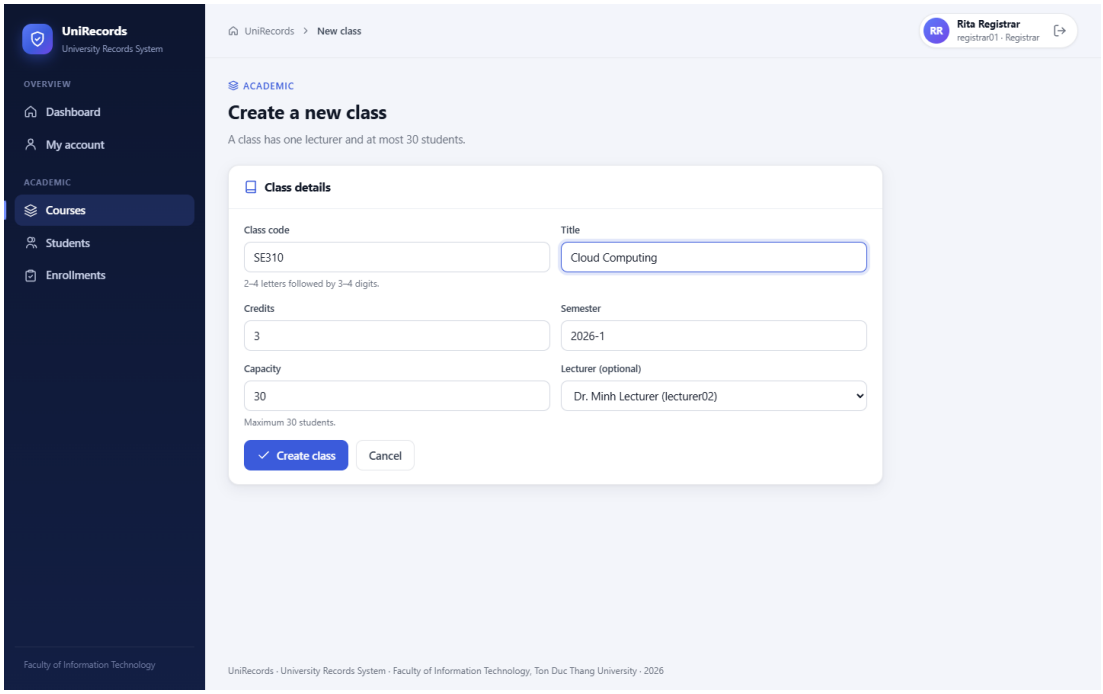


Figure A.25. *Creating a class: capacity at most 30, one optional lecturer.*

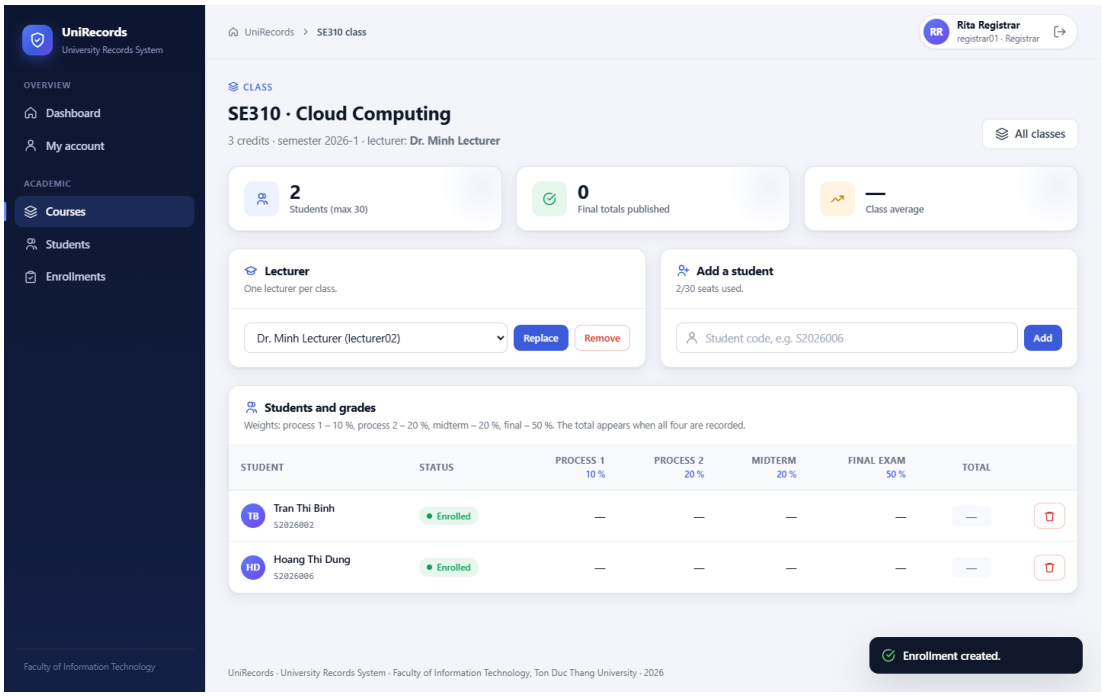


Figure A.26. *Managing class SE310: lecturer assignment, adding students, seat usage (T48).*

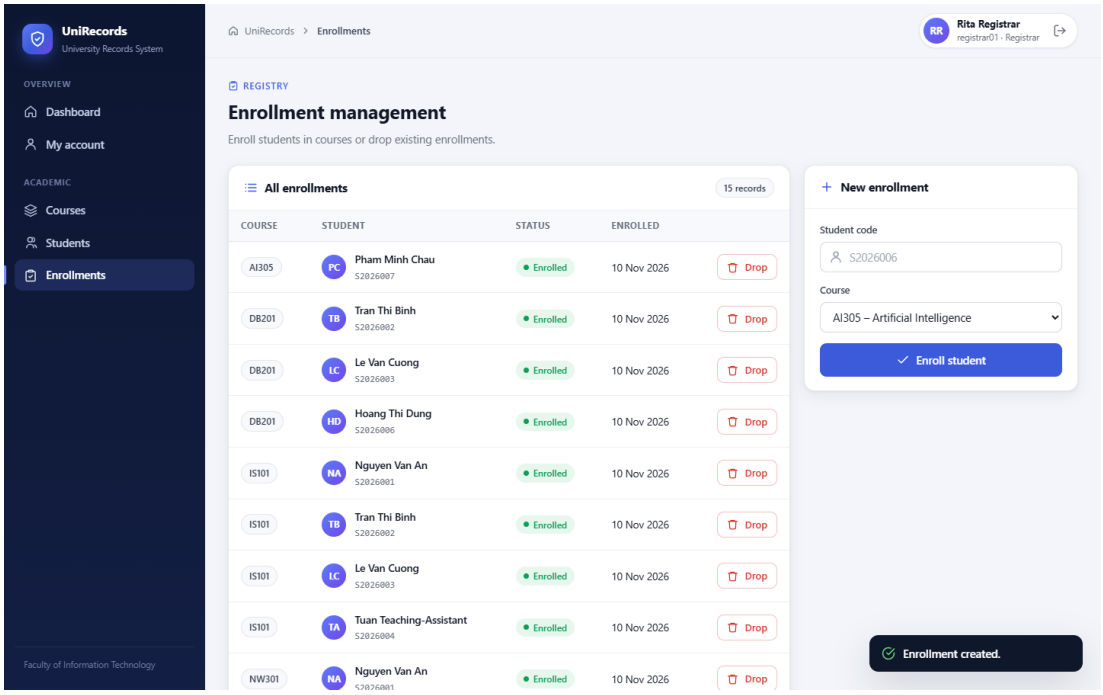


Figure A.27. *Registrar enrolls S2026003 in IS101 (T12).*

A.7 Administrator Console

The administrator's dashboard contains account statistics only, with no academic figures. The Users & roles page lists every account with role chips and its status (active, locked or disabled). It provides account actions and role assignment, and a Safeguards card states the separation-of-duties rules that the server enforces.

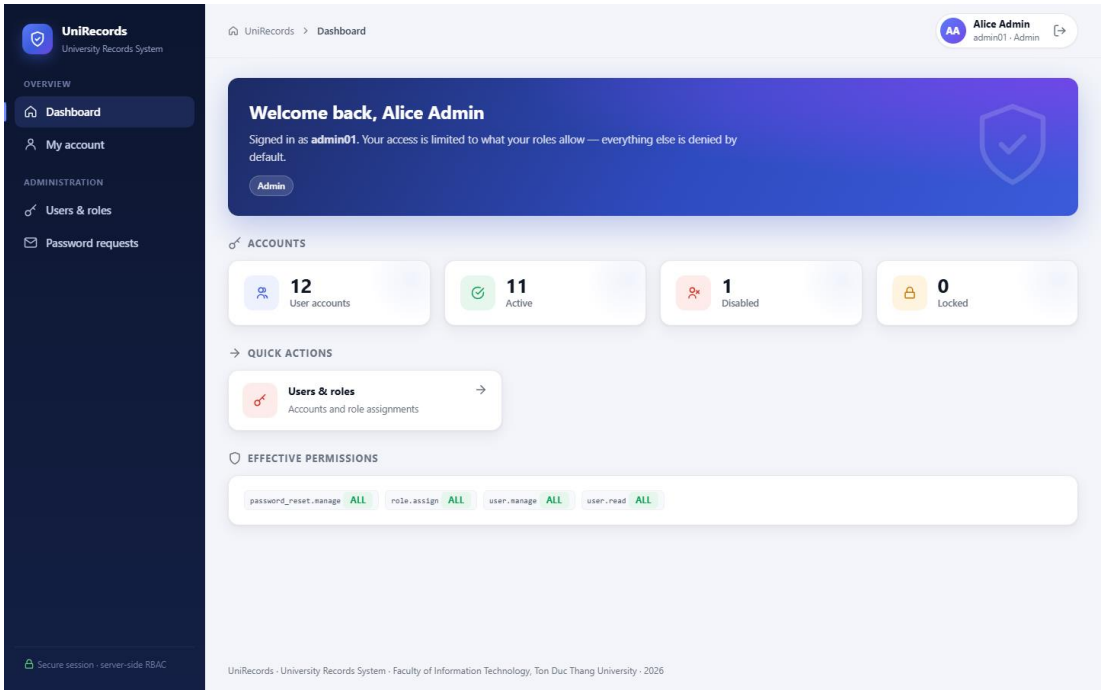


Figure A.28. *Administrator dashboard: account statistics only, no academic data.*

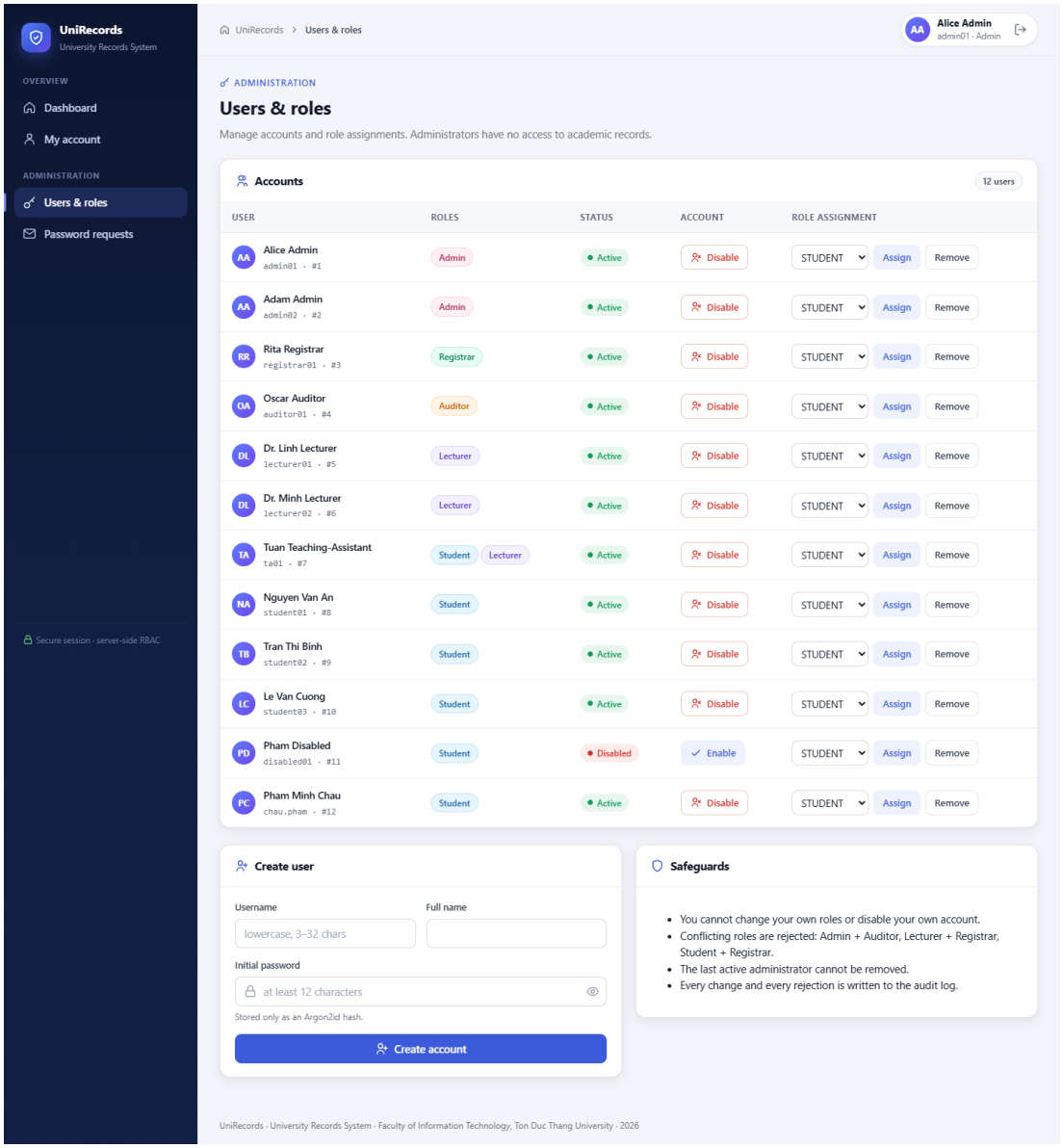


Figure A.29. *User accounts, role chips, status and role-assignment controls (T16).*

A.8 Account Self-Service and Password Reset

Every role has a My account page. Students and lecturers can rename themselves; staff names are locked, and every role can change its password after confirming the current one. A forgotten password starts with a public request that returns a reference code once. The administrator's inbox shows the requester, whether the e-mail matches the profile and the message, with Approve and Reject plus a reply. With the code, the requester reads the reply and reveals a one-time temporary password; signing in with it leads straight to a forced password change.

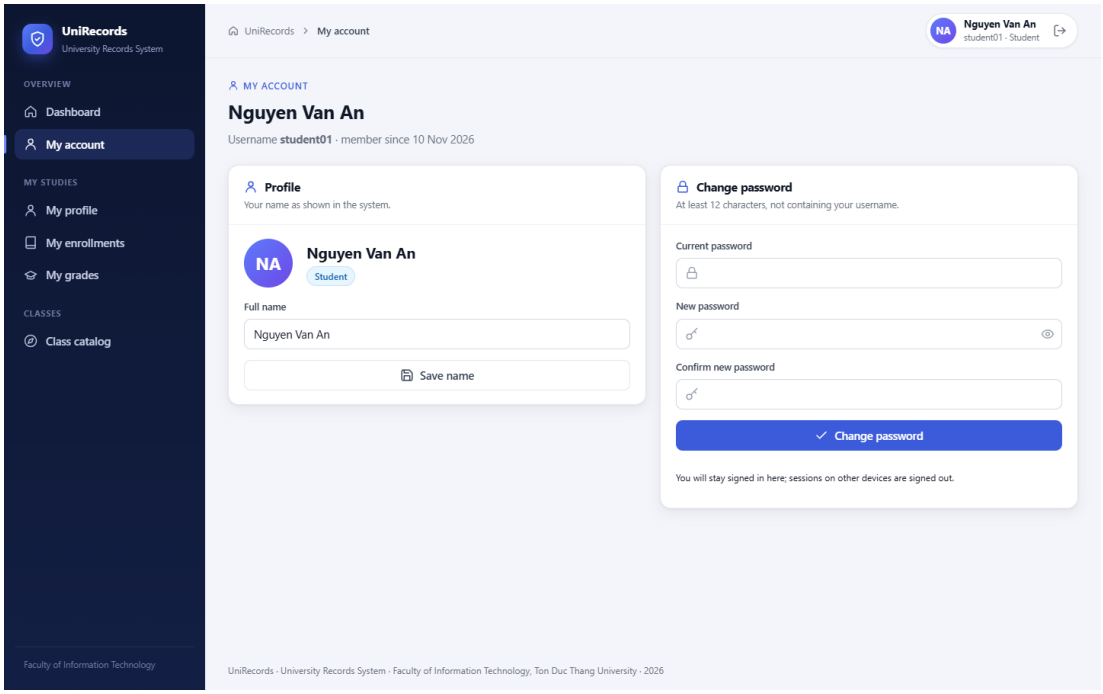


Figure A.30. *My account (student): change name and password (T53).*

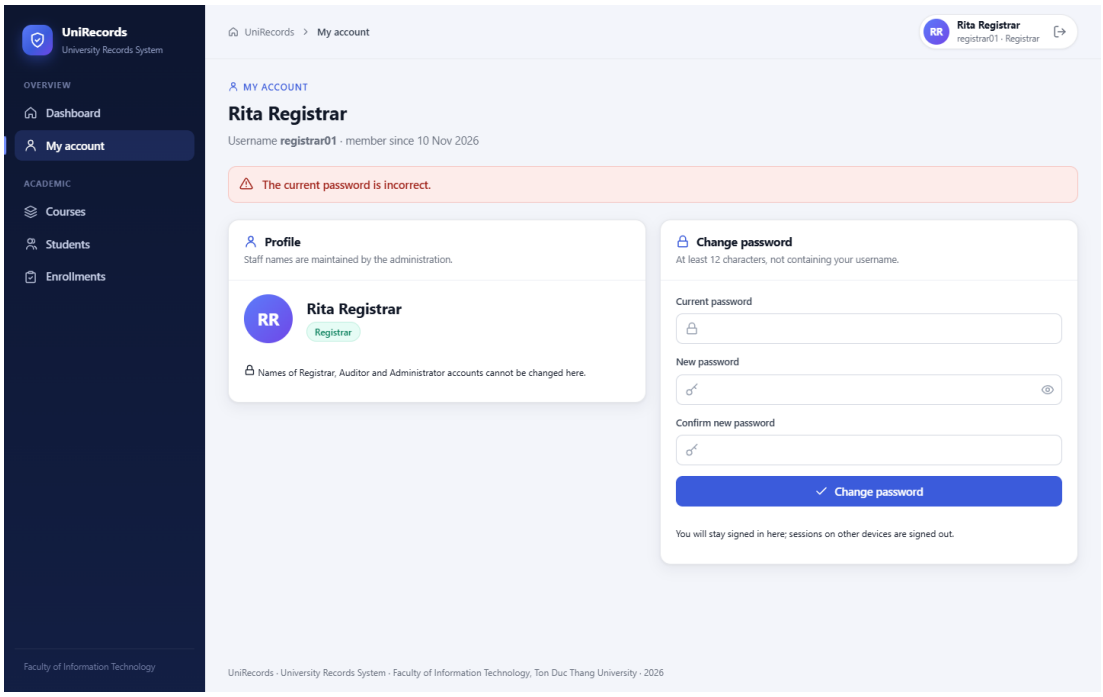


Figure A.31. *My account (registrar): name locked; a wrong current password is rejected (T53/T54).*

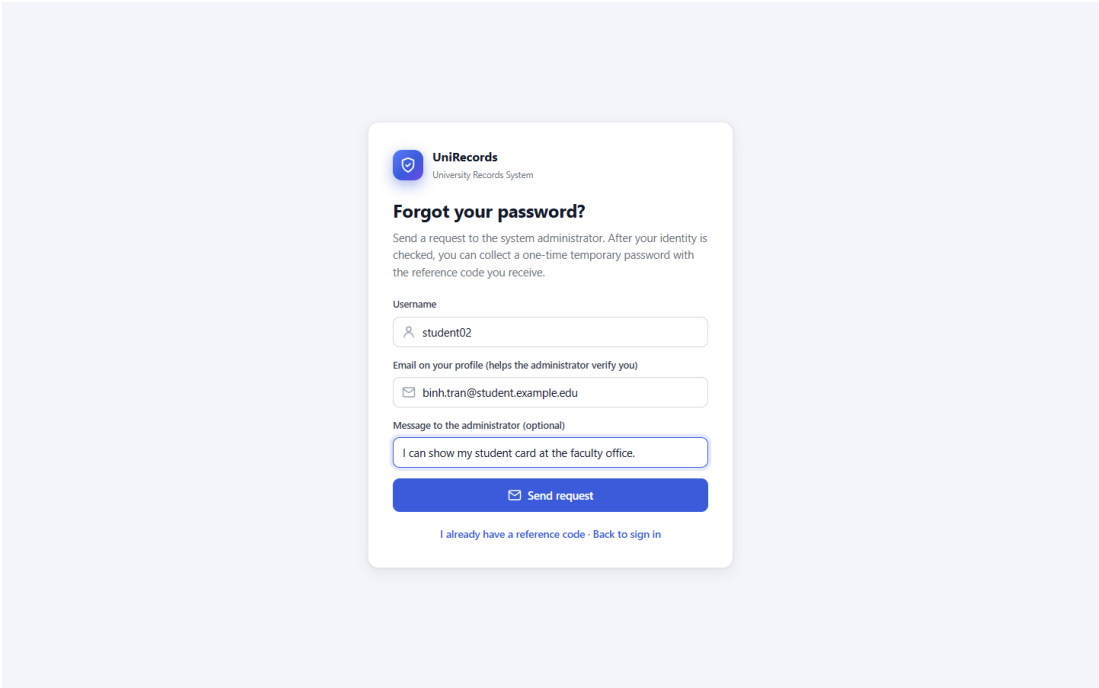


Figure A.32. *Forgot-password request form.*

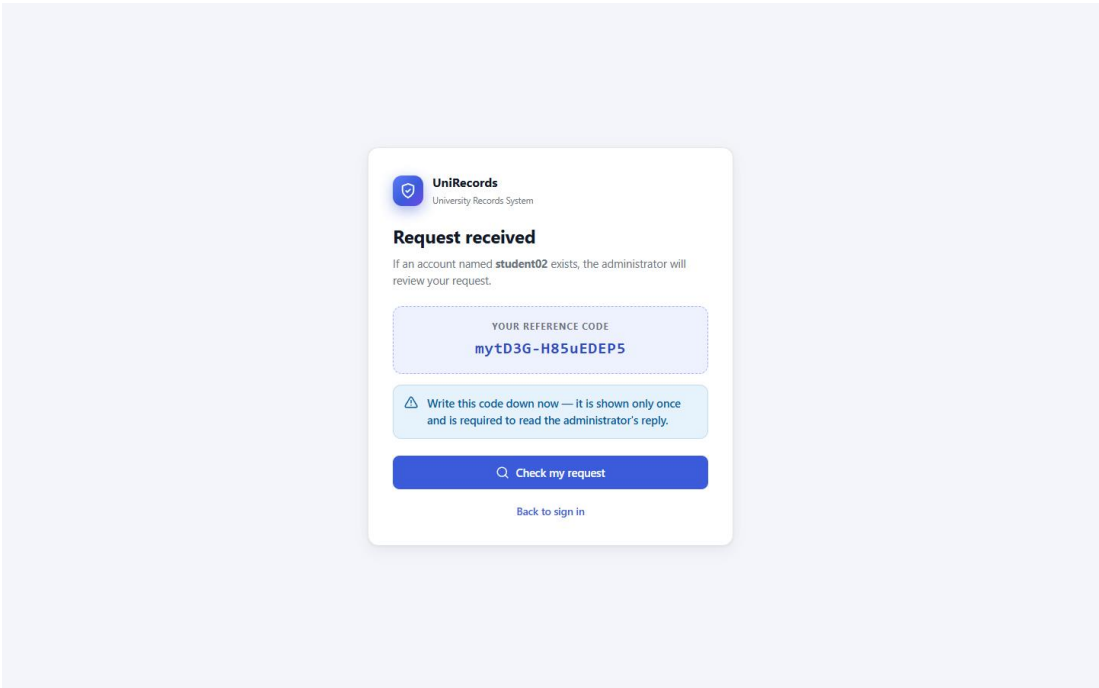


Figure A.33. *Request received: the reference code is shown once (T55).*

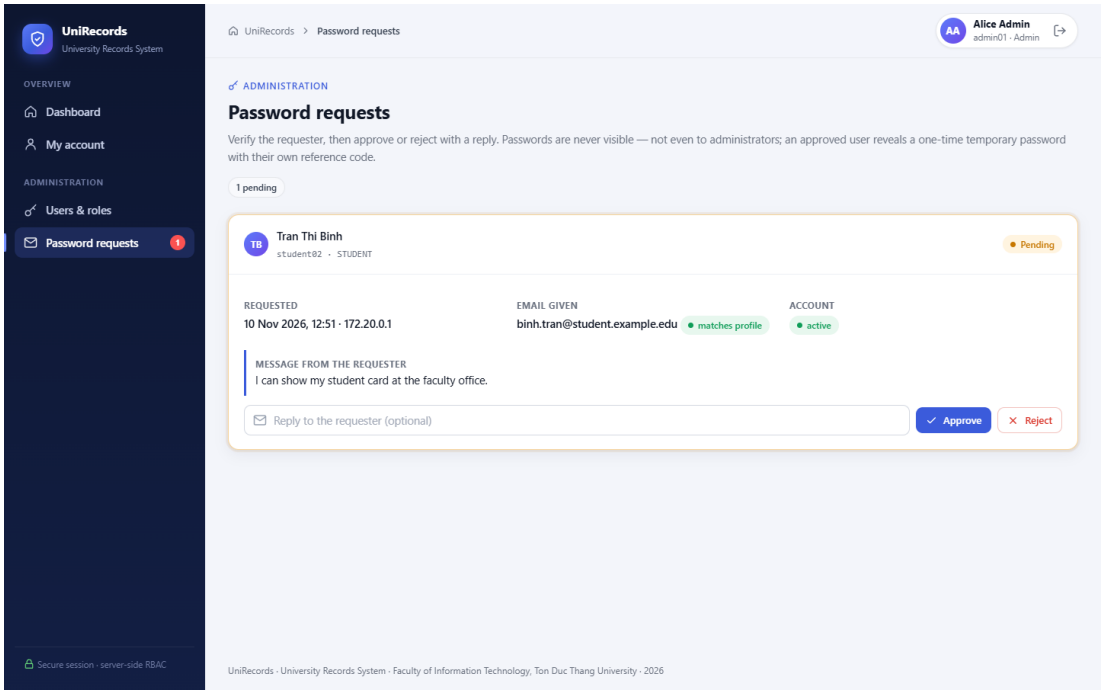


Figure A.34. *Administrator inbox: requester, e-mail check against the profile, approve or reject with a reply (T57).*

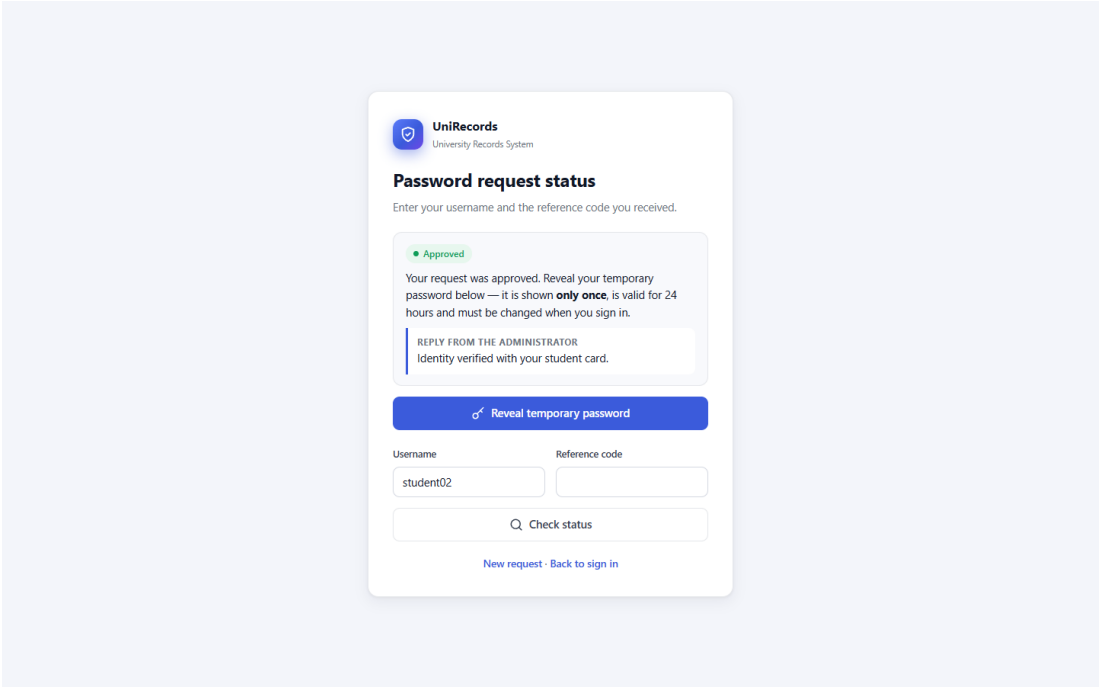


Figure A.35. *Status page: approved, with the administrator's reply (T56).*

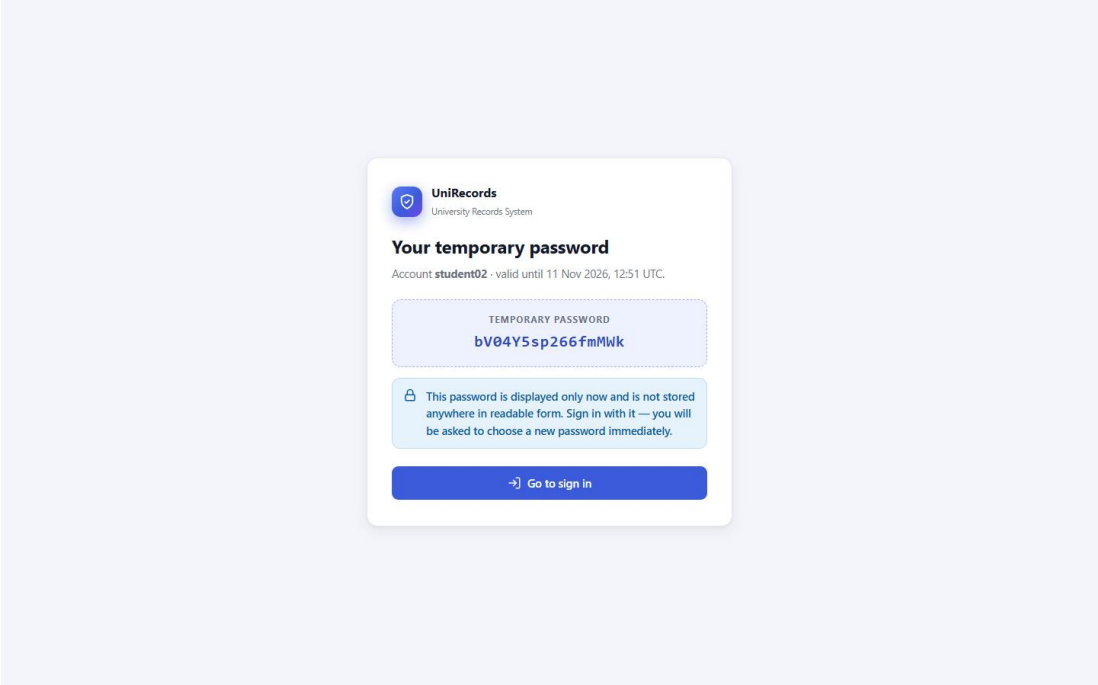


Figure A.36. *One-time temporary password, revealed only to the requester (T56).*

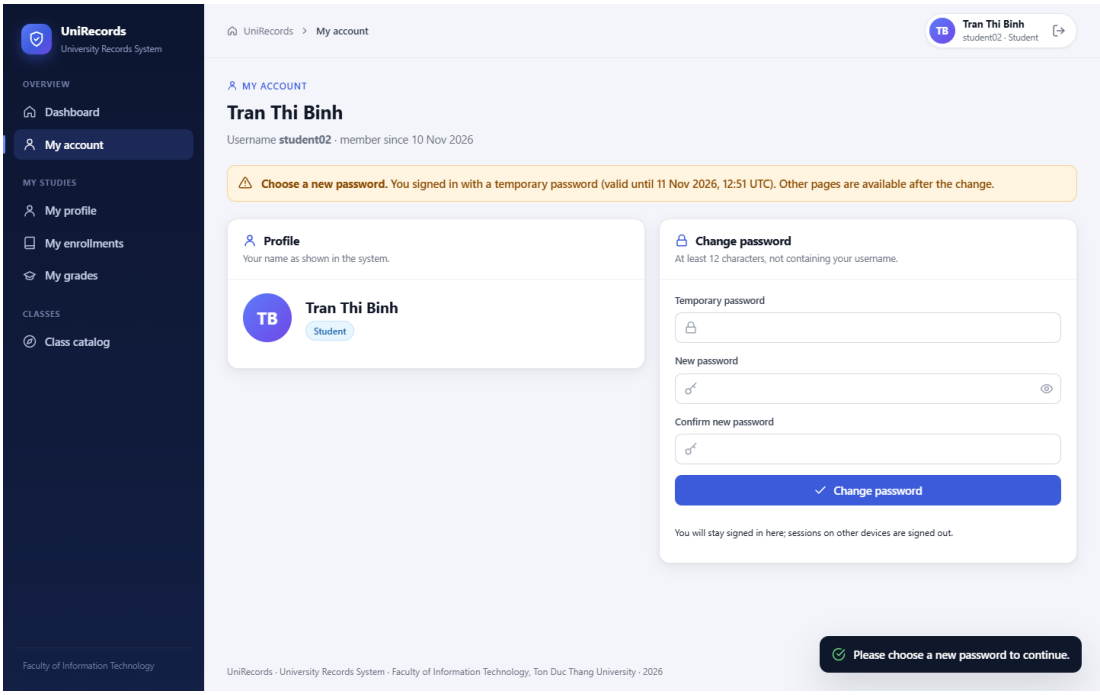


Figure A.37. *Signing in with the temporary password forces a password change (T56).*

A.9 Rejected Operations

Unauthorized requests receive a uniform Access denied page (HTTP 403) that gives no detail about the protected resource. The figures below were produced by

requests that bypass the user interface: a changed id in the URL (IDOR), or a hand-crafted POST with a valid CSRF token. They demonstrate that hiding a button is not the control; the server-side check is.

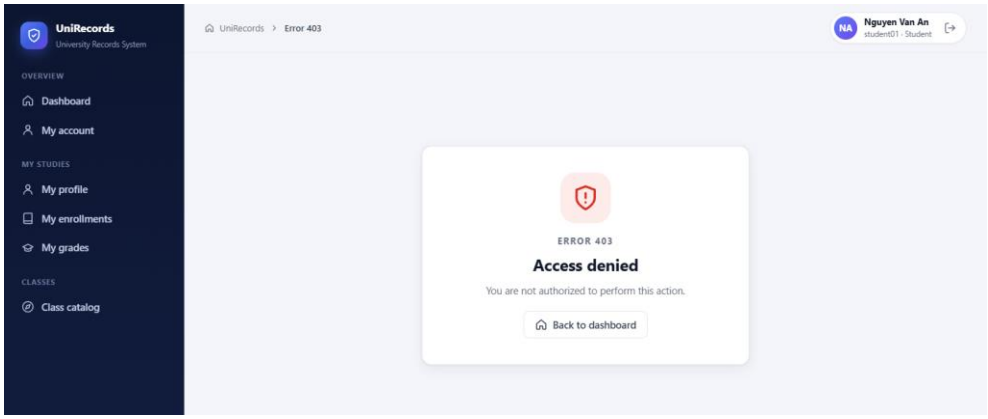


Figure A.38. *student01 changes the id in the URL to another student: HTTP 403 (T07).*

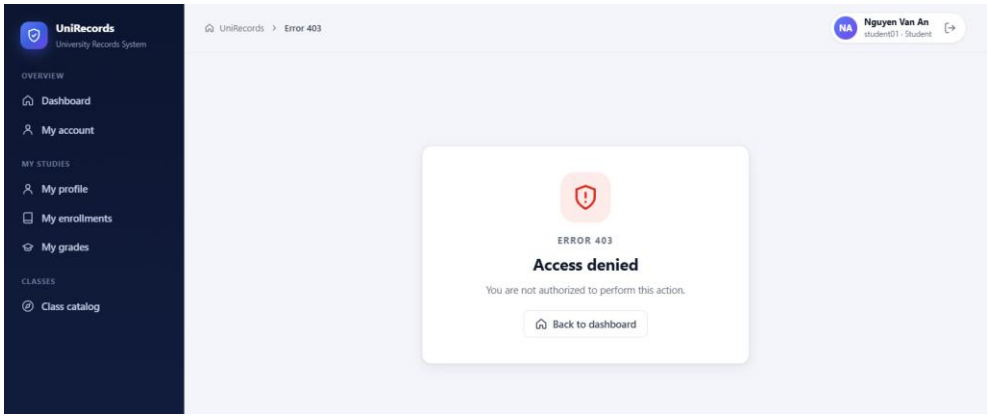


Figure A.39. *student01 sends a grade-modification request directly: HTTP 403 (T08).*

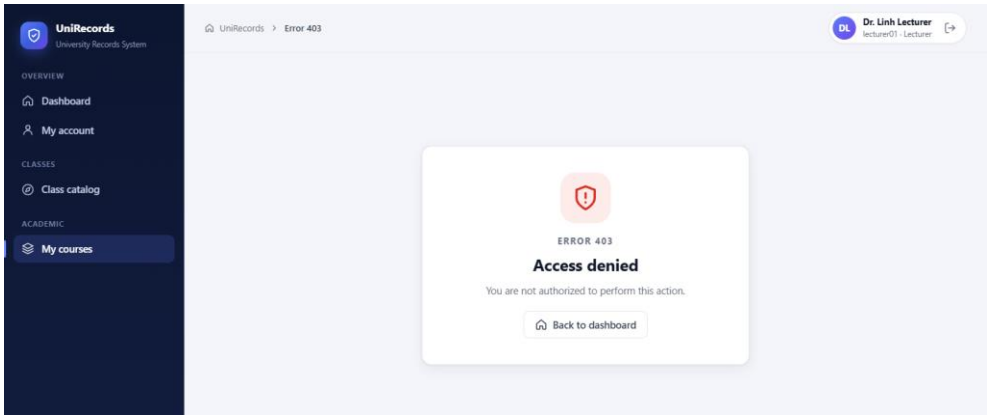


Figure A.40. *lecturer01 modifies a grade in unassigned NW301: HTTP 403 (T11).*

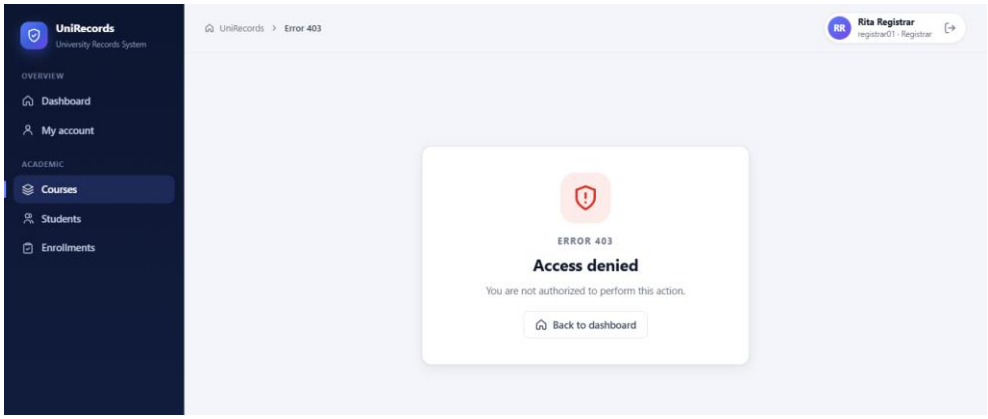


Figure A.41. *registrar01 modifies a grade: HTTP 403 (T13).*

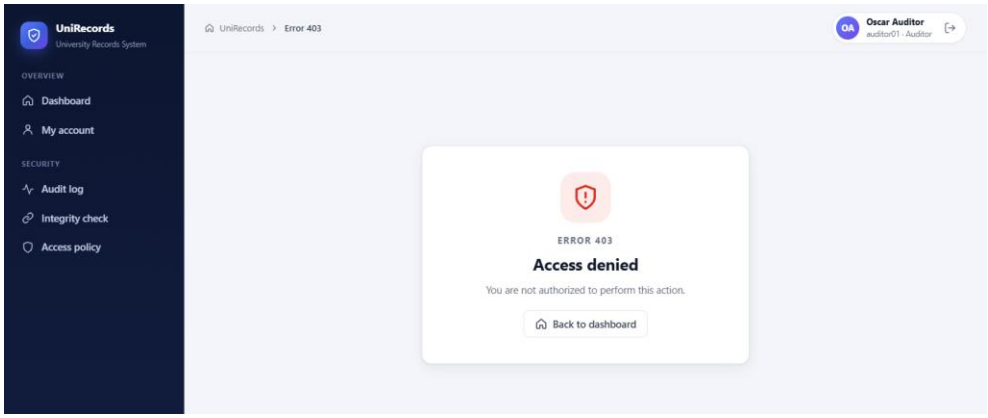


Figure A.42. *auditor01 modifies a student record: HTTP 403 (T15).*

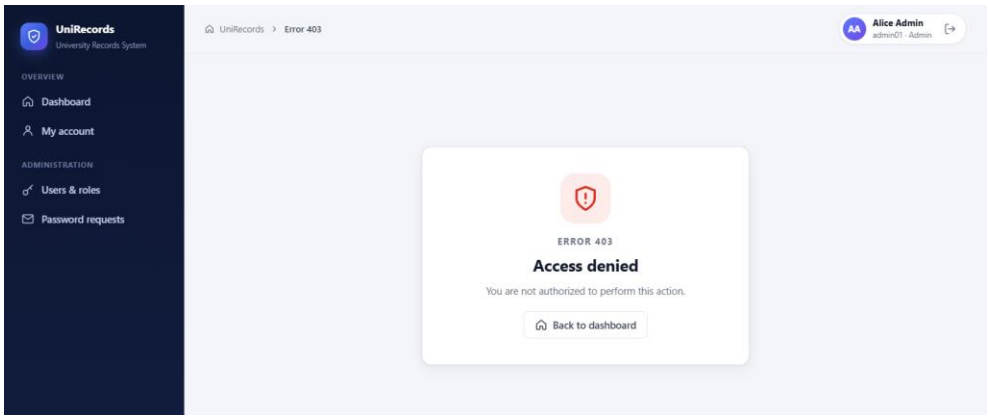


Figure A.43. *admin01 (ADMIN only) opens a grade page: HTTP 403 (T17).*

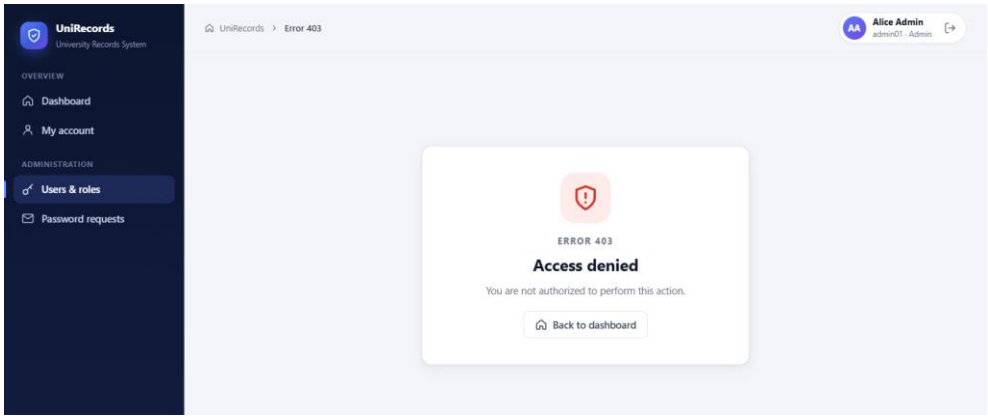


Figure A.44. *Assigning AUDITOR to an administrator violates separation of duties: HTTP 403 (T30).*

A.10 Input Handling and Session Termination

A stored script payload is rendered as text because of auto-escaping, and the CSP would block it anyway. After logout, the previous session is revoked on the server, and any request with it receives the sign-in-required page (HTTP 401).

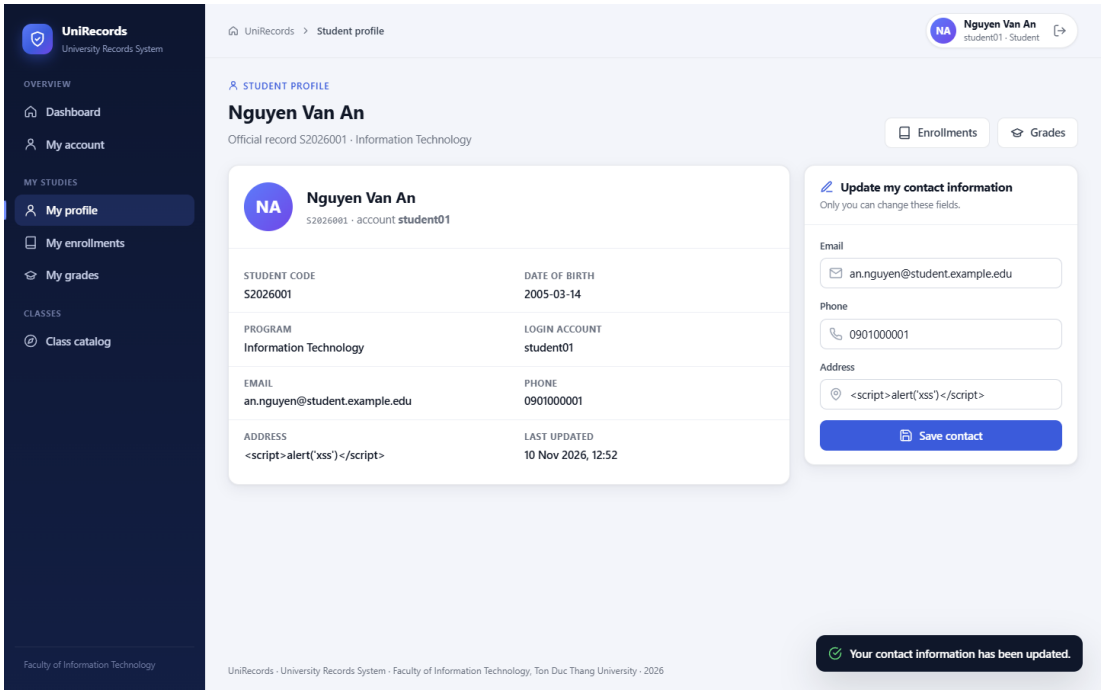


Figure A.45. *Stored <script> payload is displayed as plain text and does not execute (T19).*

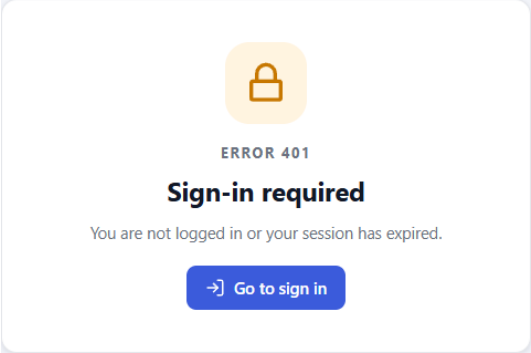


Figure A.46. *After logout the old session is rejected: HTTP 401 (T04).*

A.11 Security Auditor Console

The auditor's dashboard monitors audit volume, denied actions and failed logins. The audit log can be filtered by event type, result and actor, and the details of each entry are shown as key–value chips. The integrity page recomputes the SHA-256 hash chain in the database. The policy page renders the permission matrix, role conflicts and role assignments directly from the database, read-only.

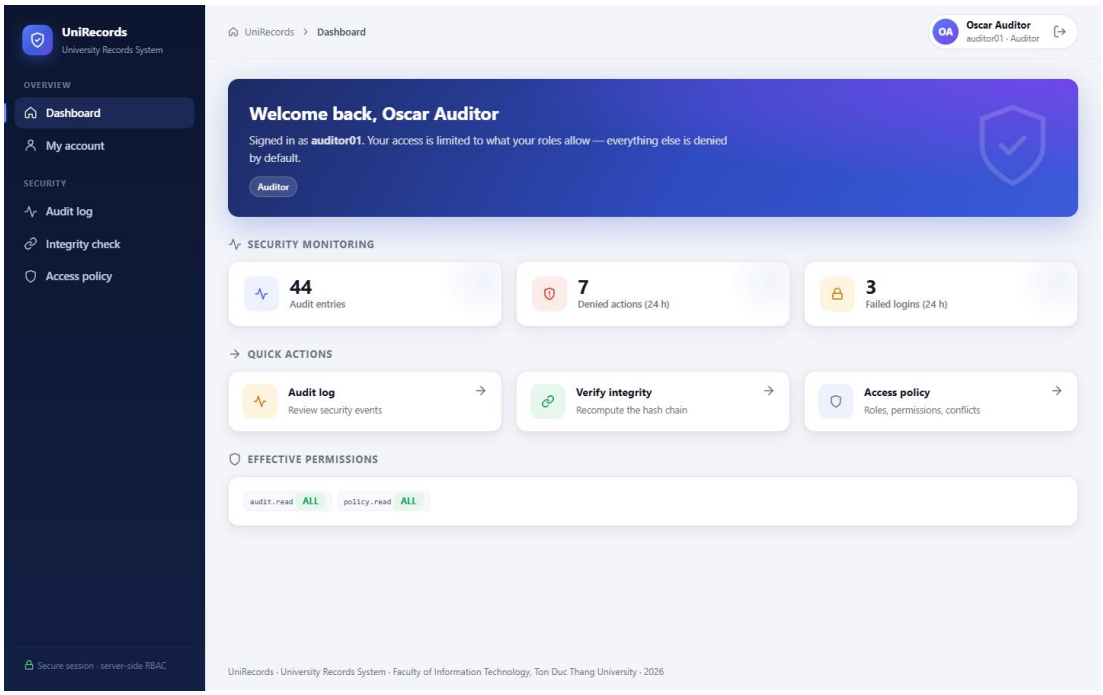


Figure A.47. *Security Auditor dashboard: audit volume, denials and failed logins.*

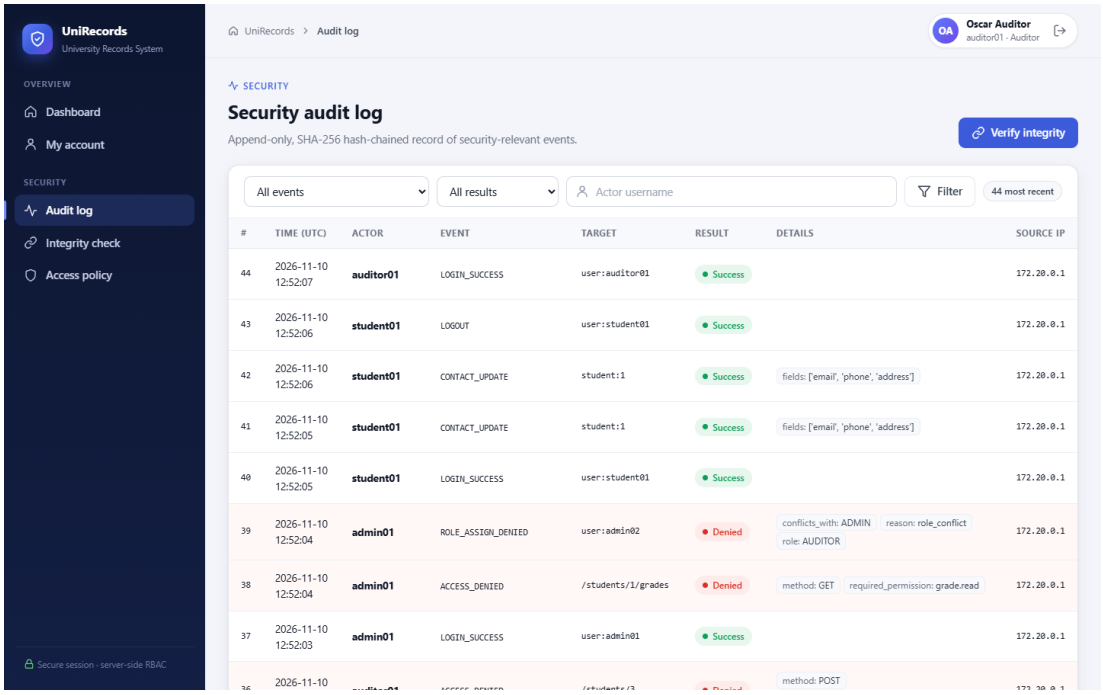


Figure A.48. *Security audit log with timestamp, actor, event, target, result and details (T14).*

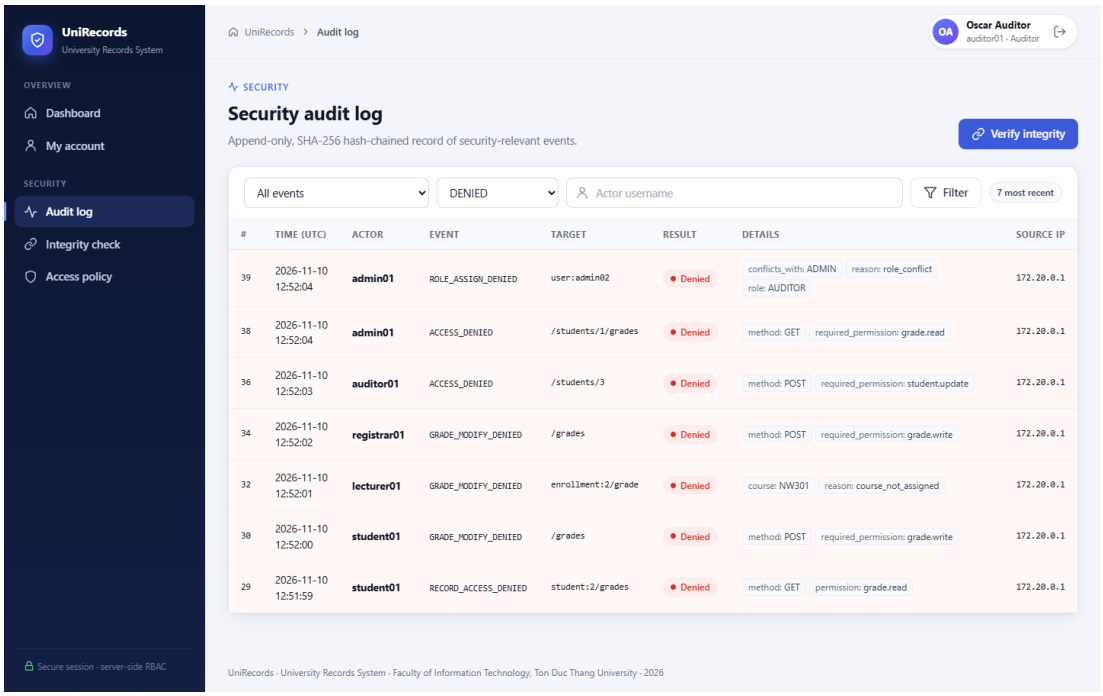


Figure A.49. *Audit log filtered on rejected actions (T20).*

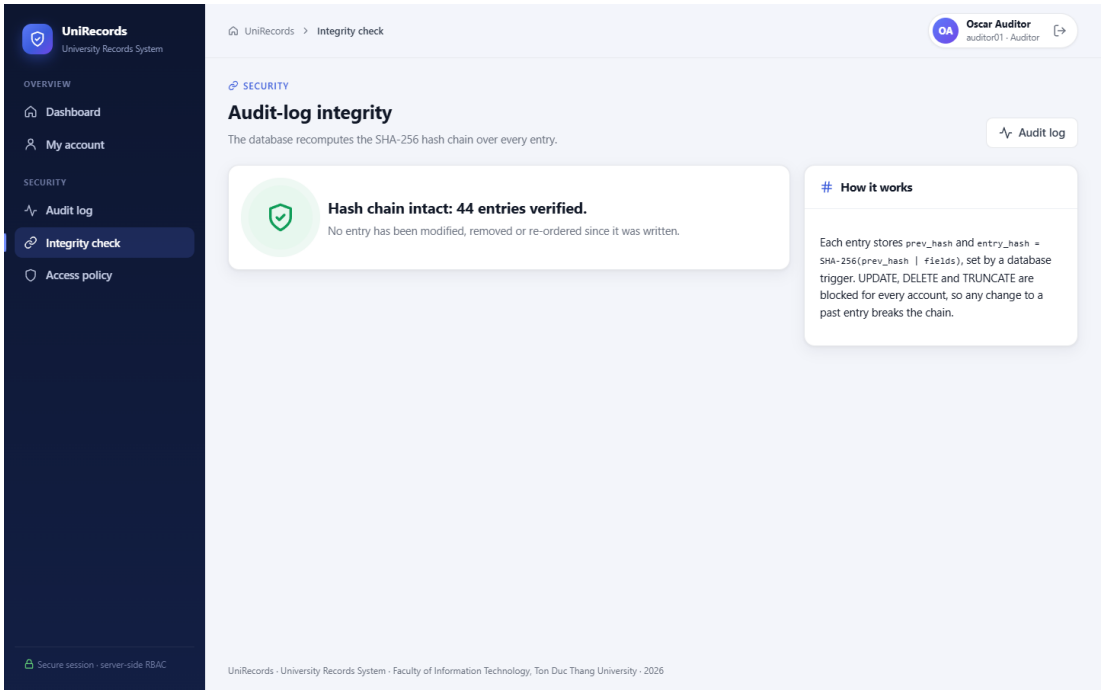


Figure A.50. *Integrity check: the SHA-256 hash chain is intact (T42).*

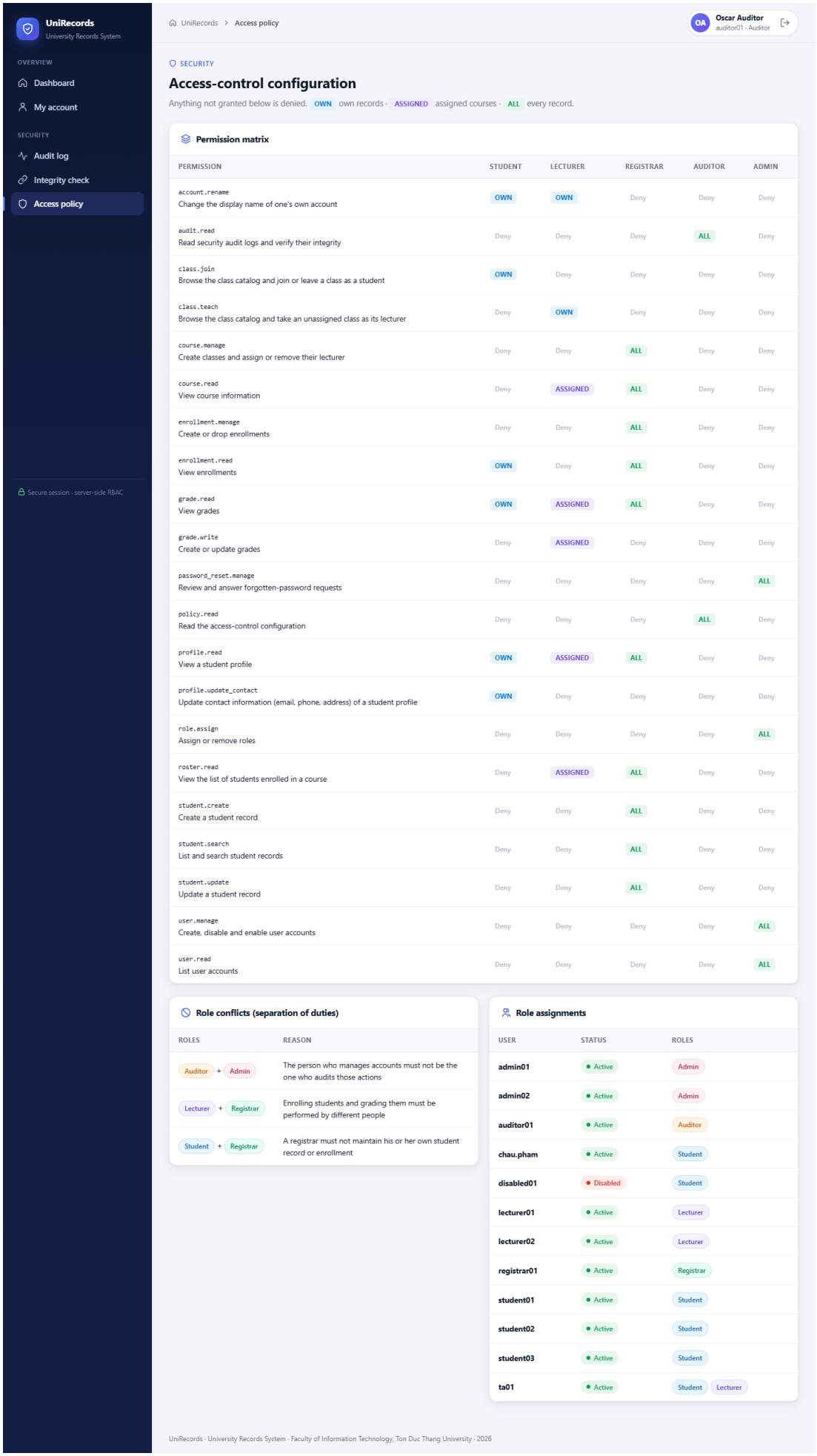

UniRecords
University Records System
OVERVIEW
Dashboard
My account
SECURITY
Audit log
Integrity check
Access policy
Secure session · server-side RBAC
UniRecords › Access policy
Oscar Auditor
auditor01 · Auditor
SECURITY
Access-control configuration
Anything not granted below is denied. OWN own records · ASSIGNED assigned courses · ALL every record.
Permission matrix
PERMISSION
STUDENT
LECTURER
REGISTRAR
AUDITOR
ADMIN
account.rename
Change the display name of one's own account
OWN
OWN
Deny
Deny
Deny
audit.read
Read security audit logs and verify their integrity
Deny
Deny
Deny
ALL
Deny
class.join
Browse the class catalog and join or leave a class as a student
OWN
Deny
Deny
Deny
Deny
class.teach
Browse the class catalog and take an unassigned class as its lecturer
Deny
OWN
Deny
Deny
Deny
course.manage
Create classes and assign or remove their lecturer
Deny
Deny
ALL
Deny
Deny
course.read
View course information
Deny
ASSIGNED
ALL
Deny
Deny
enrollment.manage
Create or drop enrollments
Deny
Deny
ALL
Deny
Deny
enrollment.read
View enrollments
OWN
Deny
ALL
Deny
Deny
grade.read
View grades
OWN
ASSIGNED
ALL
Deny
Deny
grade.write
Create or update grades
Deny
ASSIGNED
Deny
Deny
Deny
password_reset.manage
Review and answer forgotten-password requests
Deny
Deny
Deny
Deny
ALL
policy.read
Read the access-control configuration
Deny
Deny
Deny
ALL
Deny
profile.read
View a student profile
OWN
ASSIGNED
ALL
Deny
Deny
profile.update_contact
Update contact information (email, phone, address) of a student profile
OWN
Deny
Deny
Deny
Deny
role.assign
Assign or remove roles
Deny
Deny
Deny
Deny
ALL
roster.read
View the list of students enrolled in a course
Deny
ASSIGNED
ALL
Deny
Deny
student.create
Create a student record
Deny
Deny
ALL
Deny
Deny
student.search
List and search student records
Deny
Deny
ALL
Deny
Deny
student.update
Update a student record
Deny
Deny
ALL
Deny
Deny
user.manage
Create, disable and enable user accounts
Deny
Deny
Deny
Deny
ALL
user.read
List user accounts
Deny
Deny
Deny
Deny
ALL
Role conflicts (separation of duties)
ROLES
REASON
Auditor + Admin
The person who manages accounts must not be the one who audits those actions
Lecturer + Registrar
Enrolling students and grading them must be performed by different people
Student + Registrar
A registrar must not maintain his or her own student record or enrollment
Role assignments
USER
STATUS
ROLES
admin01
Active
Admin
admin02
Active
Admin
auditor01
Active
Auditor
chau.pham
Active
Student
disabled01
Disabled
Student
lecturer01
Active
Lecturer
lecturer02
Active
Lecturer
registrar01
Active
Registrar
student01
Active
Student
student02
Active
Student
student03
Active
Student
ta01
Active
Student
Lecturer
UniRecords · University Records System · Faculty of Information Technology, Ton Duc Thang University · 2026

Figure A.51. Read-only view of the access-control configuration (T14).

A.12 Database and Test Evidence

The last two figures show the content of the users table, which holds only Argon2id hashes, and the automatically generated test report.

PostgreSQL query output (connected as uni_app)

```
uni_records=> SELECT id, username, is_active, password_hash FROM users ORDER BY id;
```

id	username	is_active	password_hash
1	admin01	True	$argon2id$v=19$m=65536,t=3,p=4$70Mr8Rc+SF7a7Ddqrg3GSA$XQtVOUsPiGwDOI8Ltd4AWt8mA618ftCNhMJHmTT2KA0
2	admin02	True	$argon2id$v=19$m=65536,t=3,p=4$gKBNqgEcyouNQJBWSQK8uQ$9VHaUexwkKrYA55ks1XDdP9DavNyf9bkMFYBphnlVDQ
3	registrar01	True	$argon2id$v=19$m=65536,t=3,p=4$1VUX0Z+119jqet1CXabFig$Q/BE5feXaJwaW4OYjnXW68OArH5P+cKoJqzzimGjl5k
4	auditor01	True	$argon2id$v=19$m=65536,t=3,p=4$qejmdKx4aKAOnXfeZEg+4A$V7wSimzfxLIPgnA+9Ul1ZbRgKiq6Xn/CjyrOtjk6/Lk
5	lecturer01	True	$argon2id$v=19$m=65536,t=3,p=4$Qs0lzSFx5fqx3g75p9CYHw$CWutPbKLPKGCPVA1Go32iIFZ3GCbtQVQUMJCWgEv+kc
6	lecturer02	True	$argon2id$v=19$m=65536,t=3,p=4$Ndoh/kz8LXmpBPXtcrwH2A$5L0LtCy9Bjsz3ikQsiYPImJHOJKwqK73RAJgh+sSXEQ
7	ta01	True	$argon2id$v=19$m=65536,t=3,p=4$of8CStEkJIL9Z1BFOkDAYQ$5YVItgIWZNbr5q8ng6IfH4ix1ZroJ9TITZoWjxkiwXE
8	student01	True	$argon2id$v=19$m=65536,t=3,p=4$5Z8W9+5GhpKPhHGfJVGGkw$9XuDE7m6n4WZxp0UQhwGjBdDhuqzfmSsHpBxM+hkpI8
9	student02	True	$argon2id$v=19$m=65536,t=3,p=4$VCk++26LZwn5XEy9lk2VsA$zPHwyOZhTzol7Fa3SVTFJNGCfD9K02TsOuvWbU2t+9Q
10	student03	True	$argon2id$v=19$m=65536,t=3,p=4$3vRfv4RQ3gnHvtMcLaOtYQ$gK7cD2UIQPHc6xSckateTBDRDpCY2wKMKx3aa0tcoeo
11	disabled01	False	$argon2id$v=19$m=65536,t=3,p=4$T4/n2JQ0+oroO7HxfPtFDA$O0VTBZyAvfiJCXzxR96/WV3leE6aN8e7feoNNLNPNvs
12	chau.pham	True	$argon2id$v=19$m=65536,t=3,p=4$JPTReoP2+HUtXQVQ8xzfnw$OjcTRPiXAUvSbeykli6VffwywPqSwbJl816iT6l2WEI

Every value is an Argon2id encoded hash: no plaintext password is stored.

Figure A.52. users.password_hash contains only Argon2id hashes (T05).

University Records System - security test results

Run at 2026-11-10 19:48:40 against http://localhost:8000 — **58/58 passed** (Authentication: 16/16 · Authorization: 29/29 · Injection / validation: 6/6 · Audit log: 5/5 · Database security: 2/2)

Test ID	Category	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/Fail	Evidence
T01	Authentication	Student (student01)	Active account student01 exists	POST /login with correct username and password	Login succeeds; session cookie issued; LOGIN_SUCCESS logged	HTTP 302 -> /, dashboard 200 shows student01; LOGIN_SUCCESS logged (1 entry)	PASS	evidence/http/T01.txt
T02	Authentication	Student (student01)	Active account student01 exists	POST /login with correct username and wrong password	Login fails with a generic message; LOGIN_FAILURE logged	HTTP 401, message 'Invalid username or password.'; no cookie; LOGIN_FAILURE logged	PASS	evidence/http/T02.txt
T03	Authentication	Disabled student (disabled01)	Account disabled01 has is_active = false	POST /login with the correct password of the disabled account	Login fails (same generic message)	HTTP 401 with the generic message; no session cookie issued	PASS	evidence/http/T03.txt
T04	Authentication	Student (student01)	student01 is logged in	Copy session cookie, POST /logout, then reuse the old cookie on GET /students/<own id>/grades	Request with the old session is rejected (401)	Before logout 200; after logout the same cookie returns HTTP 401	PASS	evidence/http/T04.txt
T05	Authentication	Tester with DB access	Seeded database	SELECT username, password_hash FROM users	No plaintext passwords; all values are Argon2id hashes	11 accounts; every password_hash is an Argon2id hash (m=64MiB,t=3,p=4); no plaintext found	PASS	evidence/http/T05.txt
T06	Authorization	Student (student01)	student01 has grades in IS101 and NW301	GET /students/<own id>/grades	Allowed (200), own grades displayed	HTTP 200; grade table for S2026001 (IS101, NW301) displayed	PASS	evidence/http/T06.txt
T07	Authorization	Student (student01)	student02 (S2026002) is another student	Change id in URL: GET /students/<student02 id>, /grades and /enrollments	Rejected (403), no data of student02 returned; RECORD_ACCESS_DENIED logged	All three requests HTTP 403 without student02 data; 3 RECORD_ACCESS_DENIED entries: ['student:2', 'student:2/grades', 'student:2/enrollments']	PASS	evidence/http/T07.txt
T08	Authorization	Student (student01)	student01 is logged in (valid CSRF token)	POST /grades directly for own IS101 enrollment with final_exam=10	Rejected (403), grade unchanged; GRADE_MODIFY_DENIED logged	HTTP 403; score still (Decimal('9.00'), Decimal('8.50'), Decimal('8.00'), Decimal('8.50'), Decimal('8.45')); GRADE_MODIFY_DENIED logged	PASS	evidence/http/T08.txt
T09	Authorization	Lecturer (lecturer01)	lecturer01 is assigned to IS101	GET /courses/<IS101>/roster	Allowed (200), enrolled students listed	HTTP 200; roster lists S2026001, S2026002, S2026004 with grade forms	PASS	evidence/http/T09.txt
T10	Authorization	Lecturer (lecturer01)	lecturer01 is assigned to IS101; student01 enrolled in IS101	POST /grades (enrollment of student01 in IS101, four new components)	Allowed; components stored, total = 10/20/20/50 % weighted; GRADE_UPDATE logged	HTTP 302; components ['1.88', '3.09', '3.9', '8.23'] -> total 5.70 (was 8.45); GRADE_UPDATE logged with old and new values	PASS	evidence/http/T10.txt
T11	Authorization	Lecturer (lecturer01)	lecturer01 is NOT assigned to NW301	POST /grades for student01's NW301 enrollment	Rejected (403) and grade unchanged; GRADE_MODIFY_DENIED logged	HTTP 403; score unchanged ((Decimal('7.00'), Decimal('6.00'), Decimal('6.50'), Decimal('6.50'), Decimal('6.45'))); GRADE_MODIFY_DENIED reason=course_not_assigned	PASS	evidence/http/T11.txt
T12	Authorization	Registrar (registrar01)	registrar01 creates a new student record	POST /enrollments {student_code, course_code=IS101}	Allowed; enrollment created; ENROLLMENT_CREATE logged	HTTP 302; enrollment T4A1FFC/IS101 status ENROLLED; ENROLLMENT_CREATE logged	PASS	evidence/http/T12.txt
T13	Authorization	Registrar (registrar01)	registrar01 logged in; can view grades	POST /grades for student02's IS101 enrollment	Rejected (403), grade unchanged	View grades 200; POST /grades HTTP 403; score unchanged ((Decimal('8.00'), Decimal('7.00'), Decimal('6.50'), Decimal('7.00'), Decimal('7.00')))	PASS	evidence/http/T13.txt

Figure A.53. Automated security test results (pytest) (T01–T58).

APPENDIX B. COMPLETE SECURITY TEST RESULTS

Generated from `tests/results/results.json` (run at 2026-11-10 19:48:40, 58/58 passed). Reproduce with `python -m pytest tests`. Raw request and response traces are stored in `evidence/http/`.

Table B.1. *Complete security test results*

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T01	Student (student01)	Active account student01 exists	POST /login with correct username and password	Login succeeds; session cookie issued; LOGIN_SUCCESS logged	HTTP 302 -> /, dashboard 200 shows student01; LOGIN_SUCCESS logged (1 entry)	PASS	http/T01.txt
T02	Student (student01)	Active account student01 exists	POST /login with correct username and wrong password	Login fails with a generic message; LOGIN_FAILURE logged	HTTP 401, message 'Invalid username or password.', no cookie; LOGIN_FAILURE logged	PASS	http/T02.txt
T03	Disabled student (disabled01)	Account disabled01 has is_active = false	POST /login with the correct password of the disabled account	Login fails (same generic message)	HTTP 401 with the generic message; no session cookie issued	PASS	http/T03.txt
T04	Student (student01)	student01 is logged in	Copy session cookie, POST /logout, then reuse the old cookie on GET /students/<own id>/grades	Request with the old session is rejected (401)	Before logout 200; after logout the same cookie returns HTTP 401	PASS	http/T04.txt
T05	Tester with DB access	Seeded database	SELECT username, password_hash FROM users	No plaintext passwords; all values are Argon2id hashes	11 accounts; every password_hash is an Argon2id hash (m=64MiB,t=3,p=4); no plaintext found	PASS	http/T05.txt
T06	Student (student01)	student01 has grades in IS101 and NW301	GET /students/<own id>/grades	Allowed (200), own grades displayed	HTTP 200; grade table for S2026001 (IS101, NW301) displayed	PASS	http/T06.txt
T07	Student (student01)	student02 (S2026002) is another student	Change id in URL: GET /students/<student02 id>, /grades and /enrollments	Rejected (403), no data of student02 returned; RECORD_ACCESS_DENIED logged	All three requests HTTP 403 without student02 data; 3 RECORD_ACCESS_DENIED entries: ['student:2', 'student:2/grades', 'student:2/enrollments']	PASS	http/T07.txt
T08	Student (student01)	student01 is logged in (valid CSRF token)	POST /grades directly for own IS101 enrollment with final_exam=10	Rejected (403), grade unchanged; GRADE_MODIFY_DENIED logged	HTTP 403; score still (Decimal('9.00'), Decimal('8.50'), Decimal('8.00'), Decimal('8.50'), Decimal('8.45')); GRADE_MODIFY_DENIED logged	PASS	http/T08.txt
T09	Lecturer (lecturer01)	lecturer01 is assigned to IS101	GET /courses/<IS101>/roster	Allowed (200), enrolled students listed	HTTP 200; roster lists S2026001, S2026002, S2026004 with grade forms	PASS	http/T09.txt
T10	Lecturer (lecturer01)	lecturer01 is assigned to IS101; student01 enrolled in IS101	POST /grades {enrollment of student01 in IS101, four new components}	Allowed; components stored, total = 10/20/20/50 % weighted; GRADE_UPDATE logged	HTTP 302; components ['1.88', '3.09', '3.9', '8.23'] -> total 5.70 (was 8.45); GRADE_UPDATE logged with old and new values	PASS	http/T10.txt

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T11	Lecturer (lecturer01)	lecturer01 is NOT assigned to NW301	POST /grades for student01's NW301 enrollment	Rejected (403) and grade unchanged; GRADE_MODIFY_DENIED logged	HTTP 403; score unchanged ((Decimal('7.00'), Decimal('6.00'), Decimal('6.50'), Decimal('6.50'), Decimal('6.45'))); GRADE_MODIFY_DENIED reason=course_not_assigned	PASS	http/T11.txt
T12	Registrar (registrar01)	registrar01 creates a new student record	POST /enrollments {student_code, course_code=IS101 }	Allowed; enrollment created; ENROLLMENT_CREATE logged	HTTP 302; enrollment T4A1FFC/IS101 status ENROLLED; ENROLLMENT_CREATE logged	PASS	http/T12.txt
T13	Registrar (registrar01)	registrar01 logged in; can view grades	POST /grades for student02's IS101 enrollment	Rejected (403), grade unchanged	View grades 200; POST /grades HTTP 403; score unchanged ((Decimal('8.00'), Decimal('7.00'), Decimal('6.50'), Decimal('7.00'), Decimal('7.00')))	PASS	http/T13.txt
T14	Auditor (auditor01)	Audit log contains entries	GET /audit/logs and /audit/policy	Allowed (200)	Both pages HTTP 200; log entries and access-control matrix displayed	PASS	http/T14.txt
T15	Auditor (auditor01)	Student S2026003 exists	POST /students/<id> (update record) and POST /students/<id>/contact	Rejected (403); record unchanged	Both requests HTTP 403; student row identical before and after	PASS	http/T15.txt
T16	Admin (admin01)	admin01 creates a fresh account without roles	POST /admin/users/<id>/roles {role=AUDITOR, action=assign}	Allowed; role assigned; ROLE_ASSIGN logged	HTTP 302; role9fdfbb now has AUDITOR; ROLE_ASSIGN logged by admin01	PASS	http/T16.txt
T17	Admin (admin01)	admin01 has only the ADMIN role	GET /students/<id>/grades, /courses/<id>/roster, /students	Rejected (403)	All three requests HTTP 403; no grade data returned	PASS	http/T17.txt
T18	Anonymous	Application running	POST /login with SQL-injection strings in username and password	No unauthorized access or database error (401 generic message); users table intact	4 payloads: all HTTP 401 with the generic message, no session, no SQL error text; users table intact	PASS	http/T18.txt
T19	Student (student01) + Registrar	student01 logged in	Store <script>alert('xss')</script> in own address; view profile as student and registrar	Value stored but rendered HTML-escaped; CSP forbids inline script; script does not execute	Payload stored verbatim in DB but output as <script>... for both viewers; CSP script-src 'self' present; no executable script in the page	PASS	http/T19.txt
T20	Registrar (registrar01) + Auditor (auditor01)	registrar01 logged in	Registrar attempts POST /grades; auditor opens /audit/logs?result=DENIED	Rejection appears in audit log with timestamp, actor, event, target, result DENIED	Entry #83 2026-11-10 12:45:11 registrar01 GRADE_MODIFY_DENIED /grades DENIED {'method': 'POST', 'required_permission': 'grade.write'}; visible to auditor	PASS	http/T20.txt
T21	Anonymous	Username 'nosuchuser' does not exist	POST /login with unknown username, compare with wrong-password response	Identical status and message (no username enumeration)	Both return HTTP 401 with the same generic message	PASS	http/T21.txt
T22	Admin (admin01) + new user	admin01 creates a fresh account	5 wrong passwords, then the correct password	Account locked: correct password rejected; ACCOUNT_LOCKED logged	After 5 failures the correct password returns 401 (generic message); locked_until set; ACCOUNT_LOCKED logged	PASS	http/T22.txt

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T23	Student (student01)	None	Inspect Set-Cookie of a successful login and the sessions table	Cookie is HttpOnly and SameSite=Strict; DB stores only a SHA-256 hash of the token	Set-Cookie has HttpOnly; SameSite=Strict; raw token absent from DB, only its SHA-256 hash is stored	PASS	http/T23.txt
T24	Student (student01)	Attacker plants a known session cookie in the victim browser	Set cookie to attacker value, then log in	Server issues a new random token (no session fixation)	A new token was issued; the planted value is not a valid session (401)	PASS	http/T24.txt
T25	Admin (admin01) + new user	New user is logged in	admin01 disables the account while its session is active; user sends a request	Existing session is invalidated immediately (401); USER_DISABLE logged	Session valid before (200); after disabling, the same session gets 401; USER_DISABLE logged	PASS	http/T25.txt
T26	Lecturer (lecturer01)	lecturer01 is not assigned to NW301	GET /courses/<NW301>/ roster and /students/<id of student only in NW301 & DB201>	Roster rejected (403); profile of a student taught in DB201 allowed	NW301 roster 403; S2026003 profile 200 and only DB201 grade visible (NW301 grade hidden)	PASS	http/T26.txt
T27	Student+Lecturer (ta01)	ta01 is the lecturer of SE202 and also enrolled in SE202	POST /grades for own SE202 enrollment, then for student03's SE202 enrollment	Self-grading rejected (403, unchanged); grading another student allowed	Own grade: 403, unchanged (None), reason=self_grading; other student: 302, score set to 2.42	PASS	http/T27.txt
T28	Student+Lecturer (ta01)	ta01 holds STUDENT and LECTURER roles	GET own grades, SE202 roster (assigned), DB201 roster (not assigned)	Union of role permissions: own grades 200, SE202 200, DB201 403	200 / 200 / 403 as expected (permissions are the union, scopes still enforced)	PASS	http/T28.txt
T29	Admin (admin01)	admin01 holds ADMIN	POST /admin/users/<own id>/roles {role=REGISTRAR, action=assign}	Rejected (403): administrators cannot change their own roles	HTTP 403; admin01 still has only ADMIN; ROLE_ASSIGN_DENIED logged	PASS	http/T29.txt
T30	Admin (admin01)	admin02 holds ADMIN; ADMIN/AUDITOR is a declared role conflict	POST /admin/users/<admin02>/roles {role=AUDITOR, action=assign}	Rejected (403) - separation of duties; role not added	HTTP 403; ROLE_ASSIGN_DENIED {'role': 'AUDITOR', 'reason': 'role_conflict', 'conflicts_with': 'ADMIN'}	PASS	http/T30.txt
T31	Student (student01)	student02 has contact data	POST /students/<student02 id>/contact {email=attacker}	Rejected (403); student02 contact unchanged	HTTP 403; contact of S2026002 unchanged	PASS	http/T31.txt
T32	Lecturer (lecturer01)	lecturer01 logged in	POST /grades for an assigned course WITHOUT the CSRF token	Rejected (403), unchanged; CSRF_REJECTED logged	HTTP 403; score unchanged ((Decimal('8.00'), Decimal('7.00'), Decimal('6.50'), Decimal('7.00'), Decimal('7.00'))); CSRF_REJECTED logged	PASS	http/T32.txt
T33	Anonymous	No session	GET /students/1/grades, /audit/logs, /admin/users; POST /grades	Rejected (401) for every request	All four requests HTTP 401	PASS	http/T33.txt
T34	Admin (admin01)	admin01 has only the ADMIN role	GET /audit/logs and /audit/policy	Rejected (403): the administrator does not audit himself/herself	Both HTTP 403	PASS	http/T34.txt

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T35	Registrar (registrar01)	registrar01 logged in	GET /students?q=<SQL-injection payloads> (OR 1=1, UNION SELECT password_hash, quotes)	Payload treated as literal text: no extra rows, no hashes, no database error	All payloads HTTP 200 with 'No matching students.', no hashes or SQL errors; a normal search still works	PASS	http/T35.txt
T36	Lecturer (lecturer01)	lecturer01 assigned to IS101	POST /grades with a component = 11, -1, abc, 8.555, NaN, 1e1, 10.01, all components empty, and enrollment_id = 1 OR 1=1	Each rejected with 400 by server-side validation; grade unchanged	All 9 invalid inputs HTTP 400; score unchanged ((Decimal('8.00'), Decimal('7.00'), Decimal('6.50'), Decimal('7.00'), Decimal('7.00')))	PASS	http/T36.txt
T37	Registrar / Anonymous	Application running	Trigger errors: unknown URL, non-integer id, wrong method, oversized body, overlong search	Generic error pages; no stack trace, SQL or configuration disclosed	404/404/404/405/413/400 with generic messages; no stack trace, SQL or config	PASS	http/T37.txt
T38	Student (student01)	student01 logged in	POST own contact with invalid email, invalid phone, control characters, 250-char address	Rejected with 400; record unchanged	All 5 invalid inputs HTTP 400 with validation messages; record unchanged	PASS	http/T38.txt
T39	Application DB account and DBA	Audit log contains entries	UPDATE/DELETE /TRUNCATE audit_logs as uni_app, then UPDATE/DELETE as the DBA	uni_app: permission denied; DBA: blocked by append-only trigger; log unchanged	All 6 attempts raised errors (uni_app: 42501 permission denied; DBA: 42501 'audit_logs is append-only'); no row changed	PASS	http/T39.txt
T40	Tester with DB access	Previous tests performed logins, logouts and password-bearing requests	Search every audit entry for demo passwords, session tokens and secret keys	No password, session token or DB credential appears in the log	Scanned 85 entries: no demo password, current session token, DB password, password hash or secret key name found	PASS	http/T40.txt
T41	Tester with DB access	Full test suite has exercised all sensitive operations	Check that every required event type exists and each entry has all mandatory fields	All required event types present; timestamp, actor, event, target, result filled	All 11 required event types present; no entry lacks timestamp/actor/event/target/result	PASS	http/T41.txt
T42	Auditor (auditor01) and DBA	Audit log contains entries	Verify hash chain via /audit/verify; then, as DBA inside a rolled-back transaction, disable the trigger, alter one entry and re-verify	Intact chain verified; the tampered entry is detected; rollback leaves log intact	Intact chain verified; tampering with entry #4 detected (first_bad_id=4); after rollback chain intact again	PASS	http/T42.txt
T43	Application DB account (uni_app)	Grade of S2026002 in IS101 exists	Bypass the web layer: UPDATE grades.final_exam directly as uni_app with updated_by = registrar01 and = lecturer02 (not assigned)	grades_guard trigger rejects both (42501); score unchanged	Both direct updates rejected by the trigger (updated_by=registrar01: grade_guard: writer is not an assigned lecturer; updated_by=lecturer02: grade_guard: writer is not an assigned lecturer); score unchanged ((Decimal('8.00'), Decimal('7.00'), Decimal('6.50'), Decimal('7.00'), Decimal('7.00')))	PASS	http/T43.txt

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T44	Application DB account (uni_app)	Application connects as uni_app	As uni_app: DELETE grades/students, UPDATE role_permissions, change username, CREATE TABLE, read pg_authid; inspect role attributes	All denied (insufficient privilege); uni_app is not superuser and owns no table	All 8 statements rejected with 42501 (insufficient privilege); uni_app: no superuser/createrole/createdb/ bypassrls, owns 0 tables	PASS	http/T44.txt
T45	Anonymous → new student	Username does not exist	POST /register with valid data plus forged fields role=ADMIN and is_admin=1; log in	Account created with exactly the STUDENT role and a NEW student profile; forged fields ignored; USER_REGISTER logged	HTTP 302; roles = ['STUDENT']; new profile S2026008; login works; /admin/users 403; USER_REGISTER logged	PASS	http/T45.txt
T46	Anonymous	student01 already exists	POST /register with: short password, mismatching confirmation, password containing the username, existing username, invalid username	Each rejected with 400; no account created	All 5 invalid registrations HTTP 400 with a validation message; user count unchanged	PASS	http/T46.txt
T47	Lecturer (lecturer01) + Student (student02)	student02 enrolled in DB201 (lecturer01)	Lecturer saves 4 components; then only 3 components; student02 opens own grades page	Total = 10/20/20/50 % weighted (computed by DB); total empty while a component is missing; student sees all components, totals and overall average	Total 8.13 = 0.1×8 + 0.2×7.5 + 0.2×6 + 0.5×9.25; total NULL with a missing component; student page shows components, total and overall average	PASS	http/T47.txt
T48	Registrar / Admin / Lecturer / Student	None	POST /courses (create class) as registrar01 (capacity 30 and 31, duplicate code) and as admin01, lecturer01, student01	Registrar: created + COURSE_CREATE logged; capacity 31 and duplicate code → 400; others → 403	TDH1775 created (capacity 30, logged); capacity 31 and duplicate code 400; admin01 / lecturer01 / student01 403	PASS	http/T48.txt
T49	Registrar + Lecturers + DB account	Registrar creates a class taught by lecturer01	lecturer02 tries to take the class; direct INSERT of a 2nd lecturer as uni_app; registrar replaces the lecturer	Take-over 403; DB rejects 2nd lecturer (unique violation); replacement leaves exactly 1 lecturer	Take-over 403; direct INSERT 23505 (unique course_id); after replacement the class has exactly one lecturer (lecturer02)	PASS	http/T49.txt
T50	Registrar + DB account	Registrar creates a class with capacity 2	Enroll 3 students through the UI; then INSERT a 3rd enrollment directly as uni_app	Third enrollment rejected (400) by the app and (23514) by the capacity trigger; 2 students remain	UI: 302, 302, 400 (class full); direct INSERT 23514 'class is full (capacity 2)'; 2 active students	PASS	http/T50.txt
T51	Student (self-registered) + student01	Open class with free seats; student01 has grades in IS101	New student joins a class, joins again, leaves; student01 tries to leave graded IS101	Join 302 (ENROLLED, CLASS_JOIN logged); duplicate join 400; leave 302 (DROPPED); leaving a graded class 400	Join 302 → ENROLLED (logged); second join 400; leave 302; leaving graded IS101 400	PASS	http/T51.txt

ID	User / role	Preconditions	Request / action	Expected result	Actual result	Pass/ Fail	Evidence
T52	Lecturer (lecturer02) + Student (student01)	Registrar creates a class without a lecturer	student01 POSTs /classes/<id>/teach and /courses/<id>/lecturer; lecturer02 takes the class and opens the roster	Student requests 403; lecturer02 becomes the only lecturer (COURSE_LECTURER_ASSIGN) and can open the roster	Student 403 / 403; lecturer02 assigned (only lecturer); roster 200	PASS	http/T52.txt
T53	Student, Lecturer, Registrar, Auditor, Admin (fresh accounts)	admin01 creates one account per role	Each user POSTs /account/password with a wrong current password, then with the correct one; old password and an older second session are tested	Wrong current password 400 (PASSWORD_CHANGE_FAILURE); change 302 for every role; old password rejected, new accepted; other sessions revoked	All roles ['STUDENT', 'LECTURER', 'REGISTRAR', 'AUDITOR', 'ADMIN']: wrong current 400, change 302, other session 401, old password 401, new password 302; PASSWORD_CHANGE(_FAILURE) logged	PASS	http/T53.txt
T54	Student, Lecturer vs Registrar, Auditor, Admin	Seed accounts	POST /account/name as each role	Student and lecturer: 302, name changed (student record too); registrar, auditor, admin: 403, unchanged	{'student03': (302, True), 'lecturer02': (302, True), 'registrar01': (403, False), 'auditor01': (403, False), 'admin01': (403, False)} (status, changed); student record name follows the account name	PASS	http/T54.txt
T55	Anonymous	student02 exists; 'ghostuser' does not	POST /forgot-password for an existing and a non-existing username	Identical response with a reference code (no enumeration); request stored without any password; account unchanged	Both 200 with identical text apart from the code; table has no password column; only the SHA-256 of the code is stored; password unchanged	PASS	http/T55.txt
T56	Anonymous requester + Admin (admin01)	Fresh student account files a request	Admin approves with a reply; requester checks status, reveals the temporary password twice, logs in, opens another page, changes the password	Reveal works once (2nd → 403); temp login forced to /account; other pages redirect until changed; temp password never stored or logged in plaintext	Approve 302; status shows reply; reveal 200 once, then 403; temp login → /account, other pages 302 until changed, then 200; temp password and code absent from DB and audit log	PASS	http/T56.txt
T57	Student, Registrar, Auditor vs Admin	At least one pending request	Non-admins GET /admin/password-requests and POST an approval; admin opens the list and approves own request	Non-admins 403; admin list shows requests but no password or hash; approving own request 403	student01/registrar01/auditor01 403 (list and approve); admin list 200 without hashes or passwords; own request 403; request still PENDING	PASS	http/T57.txt
T58	Anonymous + Admin	Requests exist	Status with a wrong code; reject flow; reveal after rejection; expired temporary password	Wrong code 404 without details; rejected request shows the reply and cannot be revealed (403); expired temporary password cannot log in	Wrong code 404; rejected request shows reply, reveal 403; expired temporary password 401 (generic message)	PASS	http/T58.txt

APPENDIX C. REQUIREMENTS TRACEABILITY AND DEMO ACCOUNTS

Table C.1. *Security requirements, threats and verifying tests*

ID	Security requirement	Threats	Verified by
SR1	Username/password login; passwords stored only as Argon2id hashes.	TH1, TH9	T01, T05
SR2	Generic login errors; no account enumeration; disabled accounts cannot log in.	TH1	T02, T03, T21
SR3	Repeated failed logins lock the account temporarily.	TH1	T22
SR4	Random server-side sessions, HttpOnly/SameSite cookies, renewed at login, invalid after logout or disabling.	TH2	T04, T23, T24, T25
SR5	Every endpoint denied unless explicitly permitted; checked on the server.	TH3, TH7	T08, T13, T15, T17, T33, T34
SR6	Records accessible only to the owner, the assigned lecturer or the registrar.	TH4	T06, T07, T09, T26, T31
SR7	Grade components written only by the assigned lecturer, never for their own enrollment; total computed by the DB; enforced in the DB too.	TH7	T10, T11, T27, T43, T47
SR8	Role changes only by administrators, not on themselves, no conflicting combinations.	TH3	T16, T28, T29, T30
SR9	Parameterized SQL, server-side validation, output encoding.	TH5, TH6	T18, T19, T35, T36, T38
SR10	State-changing requests require an anti-CSRF token.	TH10	T32
SR11	Sensitive actions and rejections in an append-only, tamper-evident log without secrets.	TH8, TH9	T20, T39, T40, T41, T42
SR12	No stack trace, SQL or configuration in errors; least-privilege DB account; no secrets in the repository.	TH5, TH9	T37, T44
SR13	Self-registration creates only STUDENT accounts with a new profile and validates input.	TH11	T45, T46
SR14	Classes are created only by the registrar; one lecturer and at most 30 students per class; self-service limited to oneself.	TH11, TH3	T48, T49, T50, T51, T52
SR15	Every user can change their password (current password, policy, other sessions revoked); only students and lecturers can rename.	TH1, TH2	T53, T54
SR16	Password reset: uniform request response, secret code, admin approval, one-time 24 h temporary password visible only to the requester, forced change; no stored or admin-visible passwords.	TH12	T55, T56, T57, T58

Table C.2. *Demo accounts (password of every account: <username>@Demo2026)*

Username	Role(s)	Notes
admin01, admin02	ADMIN	Two administrators (dual control for role changes)

Username	Role(s)	Notes
registrar01	REGISTRAR	
auditor01	AUDITOR	
lecturer01	LECTURER	Assigned IS101, DB201
lecturer02	LECTURER	Assigned NW301 (AI305 has no lecturer yet)
ta01	STUDENT + LECTURER	Student S2026004 (IS101, NW301, SE202); lecturer of SE202
student01, student02, student03	STUDENT	S2026001, S2026002, S2026003
disabled01	STUDENT	Disabled account (S2026005)

REFERENCES

[1] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[2] OWASP Foundation, "Authorization Cheat Sheet," OWASP Cheat Sheet Series, 2024.

[3] OWASP Foundation, "Password Storage Cheat Sheet" and "Session Management Cheat Sheet," OWASP Cheat Sheet Series, 2024.

[4] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," OWASP Cheat Sheet Series, 2024.

[5] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, IETF, Sep. 2021.

[6] P. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST SP 800-63B, 2017 (updated 2020).

[7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, 2001.

[8] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation: Privileges, Triggers and Client Authentication," 2023.

[9] W3C, "Content Security Policy Level 3," W3C Working Draft, 2024.

[10] W3C, "Media Queries Level 5: prefers-reduced-motion," W3C Working Draft, 2023.

[11] Pallets Projects, "Flask 3.1 Documentation – Security Considerations," 2024.